

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2025

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2025

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2025

IOI China candidate team papers / National training team 2025 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2025. Tập được dàn trang bằng Adobe InDesign và có lớp văn bản đọc được cho mục lục cũng như các trang thân bài đã kiểm tra. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, bài đầu tiên của Liu Hengxi về đoạn liên tiếp cùng các bài toán liên quan, và bài thứ hai của Gou Jingyu về các phương pháp chứng minh tính lỗi trong đề OI, cùng bài thứ ba của Liu Haifeng về xử lý lân cận trên cây bằng phân rã chuỗi dài, và bài thứ tư của Xu Xiaoyang về nguyên lý chuyển vị trong một lớp bài toán quy hoạch động, cùng bài thứ năm của Chen Xinyang về cơ sở tuyến tính và các biến thể có hỗ trợ xóa.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2025*. Tập PDF gồm 168 trang; mục lục dùng số trang in của tập sách, chạy đến hơn 300 vì bản PDF được dàn trang dạng nhiều trang nội dung trên một trang PDF.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
1	Bàn về đoạn liên tiếp và các bài toán liên quan	Liu Hengxi
22	Các phương pháp chứng minh tính lời trong đề OI	Gou Jingyu
49	Một lời giải dựa trên phân rã chuỗi dài cho bài toán lân cận trên cây	Liu Haifeng
61	Ứng dụng nguyên lý chuyển vị trong một lớp bài toán quy hoạch động	Xu Xiaoyang
73	Bàn về ứng dụng cơ sở tuyến tính trong OI	Chen Xinyang
101	Tổng kết một số ý tưởng cho các bài toán liên quan tới dãy	Fan Sizhe
115	Bàn về tính chất và ứng dụng của dãy Thue–Morse	Zheng Jun
139	Bàn về nhân ma trận và phép quy giảm	Chen Nuo
150	Cách làm một số bài toán cổ điển modulo lũy thừa của số nguyên tố nhỏ	Wu Lin
161	Bàn về một lớp phương pháp duy trì thông tin trên cấu trúc cây động dễ cài đặt	Xiao Daien
169	Bàn về một lớp bài toán lân cận trên đồ thị	Jiao Siyuan
179	Bàn về một lớp thuật toán duy trì thông tin dựa trên chia để trị bằng cây đoạn	Huang Jiaxu
187	Bàn về các bài toán liên quan tới biến ngẫu nhiên liên tục	Zhou Huanyi
207	Báo cáo ra đề <i>Địa ngục vô hạn</i>	Chen Xulei
213	Báo cáo lời giải <i>Cấu trúc dữ liệu</i>	Zhu Yifan
219	Bài toán xâu dưới quan hệ tương đương đặc biệt	Zhang Mixuan
228	Bàn về hình học tính toán không sai số chính xác	Yao Xi
243	Bàn về bài toán ba lô dung lượng lớn trong OI	Fang Xintong
251	Bàn về một lớp bài toán đổi gốc trên cây	Zhao Haikun
272	Một lớp bài toán xâu trên cây được giải bằng cấu trúc dữ liệu	Li Jingrong
282	Bàn về một lớp phương pháp trích hệ số của phân thức hữu tỉ	Huang Jianheng
293	Bàn về tensor và ứng dụng trong OI	Wei Zhonghao
314	Bàn về một lớp bài toán bất đẳng thức đồng dư tuyến tính	Yang Xinhe

Bài 1. Bàn về đoạn liên tiếp và các bài toán liên quan

Liu Hengxi, Trường Trung học Trần Hải, Ninh Ba

Tóm tắt

Đoạn liên tiếp là một đối tượng thường gặp trong các kỳ thi tin học. Bài viết này chuyển một loạt bài toán liên quan đến đoạn liên tiếp về một dạng thống nhất, khảo sát các tính chất của tập khoảng, đưa ra thuật toán cho một lớp tập khoảng đặc biệt, rồi dùng các thuật toán trên tập khoảng để giải một phần các bài toán. Ngoài ra, bài viết chứng minh độ khó của một số bài toán bằng quy giảm.

1. Mở đầu

Trong các bài toán thi tin học, ta thường gặp những mô tả như “giá trị của các phần tử tạo thành một khoảng” hoặc “ $r - l = \max - \min$ ”. Những bài toán kiểu này thường liên quan đến đoạn liên tiếp. Khi làm bài [CCPC Final 2021] *Tree Partition*, tác giả nhận thấy còn có cách làm tốt hơn, từ đó suy nghĩ về trường hợp tổng quát hơn và thu được một số kết quả trong bài viết này.

Tác giả mong rằng thông qua bài viết này, người đọc có thể hiểu bản chất của bài toán đoạn liên tiếp, nắm được một số tính chất của đoạn liên tiếp, và biết cách dùng thuật toán trên tập khoảng để giải một số bài toán.

2. Quy ước

Nếu không nói rõ, các số xuất hiện đều là số nguyên.

Với $l, r \in \mathbb{Z}$, $l \leq r$, khoảng $[l, r]$ biểu thị tập các số nguyên giữa l và r , tức là

$$[l, r] = \{i \mid i \in \mathbb{Z}, l \leq i \leq r\}.$$

Khi $l < r$, gọi $[l, r]$ là một khoảng không tầm thường. Độ dài của khoảng $[l, r]$ là số phần tử trong nó, tức là $r - l + 1$.

Với một hoán vị p , ký hiệu p^{-1} là hoán vị nghịch đảo của p , tức là $p_{p_i}^{-1} = i$.

Với một hoán vị p , ký hiệu $p_{[l,r]}$ là tập tạo bởi $p_l, \dots, p_r$, tức là

$$p_{[l,r]} = \{p_i \mid i \in [l, r]\}.$$

Nếu không nói rõ, kích thước của hoán vị p là n , chỉ số của p nằm trong $[1, n]$, và khoảng $[l, r]$ thỏa $1 \leq l \leq r \leq n$.

Ký hiệu U là tập tất cả các khoảng được chứa trong $[1, n]$, tức là

$$U = \{[l, r] \mid 1 \leq l \leq r \leq n\}.$$

Ký hiệu 2^A là tập lũy thừa của tập A , tức là $2^A = \{B \mid B \subseteq A\}$.

3. Đoạn liên tiếp

3.1. Dẫn nhập

Định nghĩa 3.1 (đoạn liên tiếp của hoán vị). Với một hoán vị p , nếu khoảng $[l, r]$ thỏa $p_{[l,r]} \in U$, thì gọi $[l, r]$ là một đoạn liên tiếp của p .

Trong các kỳ thi tin học, ta thường gặp các bài toán về đoạn liên tiếp và các mở rộng liên quan. Mở rộng có thể là thay khoảng của hoán vị bằng cây con liên thông của cây, hoặc thay một hoán vị bằng nhiều hoán vị. Những mở rộng này thường đều có thể biểu diễn dưới một dạng thống nhất.

3.2. Dạng thống nhất

Giả sử có m cấu trúc và n phần tử. Gọi tập tạo bởi n phần tử là A ; để tiện, có thể xem $A = [1, n]$. Mỗi cấu trúc chứa n phần tử, và trên mỗi cấu trúc ta định nghĩa một tập con của A có “liên thông” hay không. Ta cần tìm thông tin về các tập con của A đồng thời “liên thông” trên cả m cấu trúc.

Xem các tập con của A đồng thời “liên thông” trên m cấu trúc là các đoạn liên tiếp suy rộng.

Việc định nghĩa mỗi tập con của A có “liên thông” trên một cấu trúc hay không tương đương với việc cấu trúc đó có một tập liên thông $C \subseteq 2^A$ chứa tất cả các tập con “liên thông”.

Do đó bài toán cũng có thể phát biểu là: có n phần tử, gọi tập của chúng là A , và có m tập con $C_1, \dots, C_m$ của 2^A ; ta cần tìm thông tin về

$$\bigcap_i C_i.$$

Trong các bài toán thi tin học, những cấu trúc thường gặp gồm:

- Hoán vị, tức dãy chuyển: tập liên thông của hoán vị p thường được định nghĩa là $\{p_{[l,r]} \mid [l,r] \in U\}$, tức là các khoảng của p . Tập liên thông này cũng chính là tập liên thông của dãy chuyển $T = (V, E)$, $V = [1, n]$, $E = \{(p_i, p_{i+1}) \mid i \in [1, n-1]\}$.
- Cây: với cây $T = (V, E)$, $V = A$, tập liên thông thường được định nghĩa là

$$\{S \mid S \subseteq V, T[S] \text{ là đồ thị liên thông}\}.$$

Định nghĩa này tương đương với

$$\{S \mid S \subseteq V, |\{(u, v) \in E \mid u, v \in S\}| = |S| - 1\}.$$

Ở đây $G[S]$ là đồ thị con cảm sinh bởi tập đỉnh S .

- Đồ thị: với đồ thị $G = (V, E)$, $V = A$, tập liên thông thường được định nghĩa là $\{S \mid S \subseteq V, G[S] \text{ là đồ thị liên thông}\}$.

Những tổ hợp cấu trúc có thể xuất hiện gồm:

- hai dãy chuyển, tức đoạn liên tiếp của hoán vị;
- nhiều dãy chuyển hơn;
- dãy chuyển và cây;
- dãy chuyển và nhiều cây;
- dãy chuyển và đồ thị;
- hai cây.

Với tổ hợp không chứa dãy chuyển, chẳng hạn hai cây, việc đếm số đoạn liên tiếp đã khá khó.

Với tổ hợp có chứa dãy chuyển, số đoạn liên tiếp không vượt quá $O(n^2)$, và bài toán tương ứng cũng tương đối dễ hơn. Ta có thể đánh số lại các phần tử thành $1, 2, \dots, n$ theo thứ tự trên một dãy chuyển. Sau khi đánh số lại, mọi đoạn liên tiếp đều là khoảng, vì vậy ta cần khảo sát tính chất của các tập khoảng.

4. Tập khoảng

Định nghĩa 4.1 (chính quy). Nếu họ tập S thỏa

$$S \subseteq U, \quad [1, n] \in S, \quad \forall i \in [1, n], \{i\} \in S,$$

thì gọi S là chính quy. Ở đây họ tập là một tập có phần tử là các tập.

Định nghĩa 4.2 (đóng loại I). Nếu họ tập chính quy S thỏa

$$\forall A, B \in S, \quad A \cap B \neq \emptyset \Rightarrow A \cup B \in S,$$

thì gọi S là đóng loại I.

Định nghĩa 4.3 (đóng loại II). Nếu họ tập chính quy S thỏa

$$\forall A, B \in S, \quad A \cap B \neq \emptyset \Rightarrow A \cap B \in S,$$

thì gọi S là đóng loại II.

Định nghĩa 4.4 (đóng loại III). Nếu họ tập chính quy S thỏa

$$\forall A, B \in S, \quad A \cap B \neq \emptyset \wedge A \not\subseteq B \wedge B \not\subseteq A \Rightarrow A \setminus B \in S,$$

thì gọi S là đóng loại III.

Ta có ngay các kết luận sau từ định nghĩa:

1. Giao của một số họ tập chính quy là họ tập chính quy.
2. Giao của một số họ tập đóng loại I là họ tập đóng loại I.
3. Giao của một số họ tập đóng loại II là họ tập đóng loại II.
4. Giao của một số họ tập đóng loại III là họ tập đóng loại III.

4.1. Tập khoảng đóng loại I và II

Định nghĩa 4.5 (khoảng cực tiểu). Giả sử tập khoảng S là tập khoảng đóng loại I và II. Với $i \in [1, n-1]$, nếu khoảng $[l, r]$ chứa $[i, i+1]$, thuộc S , và mọi khoảng như vậy đều chứa $[l, r]$, tức là

$$[l, r] \supseteq [i, i+1], \quad [l, r] \in S, \quad \forall [l', r'] \in S, \quad [l', r'] \supseteq [i, i+1] \Rightarrow [l', r'] \supseteq [l, r],$$

thì gọi $[l, r]$ là khoảng cực tiểu của S chứa $[i, i+1]$, ký hiệu

$$M_S(i) = [ml_S(i), mr_S(i)].$$

Dãy gồm $n-1$ khoảng cực tiểu được gọi là dãy khoảng cực tiểu của S , ký hiệu M_S .

Định lý 4.1. Giả sử tập khoảng S đóng loại I và II. Với mọi $i \in [1, n-1]$, tồn tại khoảng cực tiểu của S chứa $[i, i+1]$.

Chứng minh. Với S đóng loại I và II và $i \in [1, n-1]$, lấy

$$[l, r] = \bigcap_{[x, y] \supseteq [i, i+1] \wedge [x, y] \in S} [x, y].$$

Vì $[1, n] \in S$, về phải có ít nhất một khoảng được lấy giao. Rõ ràng $[l, r] \supseteq [i, i+1]$, và nếu $[l', r'] \supseteq [i, i+1]$ thì $[l', r'] \supseteq [l, r]$. Chỉ cần chứng minh $[l, r] \in S$. Lần lượt áp dụng tính đóng loại II của S cho các khoảng ở về phải, suy ra $[l, r] \in S$. $\dashv$

Định lý 4.2. Giả sử tập khoảng S đóng loại I và II. Với khoảng không tầm thường $[l, r]$ ($l < r$), điều kiện cần và đủ để $[l, r] \in S$ là

$$\forall i \in [l, r-1], \quad M_S(i) \subseteq [l, r].$$

Chứng minh. Xét tập khoảng S đóng loại I và II và khoảng không tầm thường $[l, r]$.

Tính đủ: nếu $\forall i \in [l, r-1], M_S(i) \subseteq [l, r]$, thì

$$[l, r] = \bigcup_{i=l}^{r-1} M_S(i).$$

Với $i \in [l+1, r-1]$, khi lấy hợp $M_S(i)$ với $\bigcup_{j=l}^{i-1} M_S(j)$, ta có $i \in M_S(i)$ và $i \in \bigcup_{j=l}^{i-1} M_S(j)$. Vì vậy có thể lần lượt áp dụng tính đóng loại I của S , suy ra $[l, r] \in S$.

Tính cần: nếu $[l, r] \in S$, theo định nghĩa khoảng cực tiểu, với mọi $i \in [l, r-1]$, rõ ràng $[l, r] \supseteq [i, i+1]$, nên $M_S(i) \subseteq [l, r]$. $\dashv$

Từ định lý trên, một tập khoảng đóng loại I và II có thể được xác định bởi $n-1$ khoảng cực tiểu.

4.2. Tập khoảng đóng loại I, II và III

Bổ đề 4.1. Giả sử tập khoảng S đóng loại I, II và III. Nếu i, j ($1 \leq i < j \leq n - 1$) thỏa

$$i + 1 \leq ml_S(j) \leq mr_S(i) \leq j,$$

thì $ml_S(j) = i + 1$ và $mr_S(i) = j$.

Chứng minh. Giả sử $i + 1 < ml_S(j) \leq mr_S(i) \leq j$. Theo định nghĩa khoảng cực tiểu, $[ml_S(i), mr_S(i)]$ và $[ml_S(j), mr_S(j)]$ đều thuộc S . Vì S đóng loại III,

$$[ml_S(i), mr_S(i)] \setminus [ml_S(j), mr_S(j)] = [ml_S(i), ml_S(j) - 1] \in S.$$

Khoảng này chứa $[i, i + 1]$, mâu thuẫn với việc $[ml_S(i), mr_S(i)]$ là khoảng cực tiểu chứa $[i, i + 1]$. Do đó $ml_S(j) = i + 1$. Tương tự, $mr_S(i) = j$. $\dashv$

Lấy T là tập các chỉ số i sao cho khoảng cực tiểu chứa $[i, i + 1]$ không bị một khoảng cực tiểu khác chứa thật:

$$T = \{i \in [1, n - 1] \mid \neg(\exists j \in [1, n - 1], M_S(i) \subsetneq M_S(j))\}.$$

Bổ đề 4.2. Với $i \in [1, n - 1]$, nếu tồn tại $t \in T$ sao cho $M_S(i) \supseteq [t, t + 1]$, thì $i \in T$.

Chứng minh. Giả sử $i \notin T$. Khi đó tồn tại $j \in [1, n - 1]$ sao cho $M_S(i) \subsetneq M_S(j)$. Vì $M_S(t)$ là khoảng cực tiểu chứa $[t, t + 1]$ và $M_S(i) \supseteq [t, t + 1]$, ta có $M_S(t) \subseteq M_S(i)$. Do $M_S(i) \subsetneq M_S(j)$, suy ra $M_S(t) \subseteq M_S(i) \subsetneq M_S(j)$, mâu thuẫn với $t \in T$. $\dashv$

Bổ đề 4.3. Với mọi $i \in [1, n - 1]$, tồn tại $t \in T$ sao cho $M_S(t)$ chứa $[i, i + 1]$.

Chứng minh. Với $i \in [1, n - 1]$, trong các $M_S(j)$ chứa $[i, i + 1]$, chọn khoảng có độ dài lớn nhất và gọi là $M_S(t)$. Nếu $M_S(t)$ bị một $M_S(u)$ chứa thật, thì $M_S(u)$ cũng chứa $[i, i + 1]$ và có độ dài lớn hơn, mâu thuẫn. Vì vậy $M_S(t)$ không bị khoảng cực tiểu khác chứa thật, tức là $t \in T$. $\dashv$

Bổ đề 4.4. Nếu tồn tại $i, j \in T$, $i \neq j$, sao cho $M_S(i) = M_S(j)$, thì $\forall t \in T$, $M_S(t) = [1, n]$.

Chứng minh. Giả sử $i, j \in T$, $i \neq j$, và $M_S(i) = M_S(j)$. Nếu $ml_S(i) \neq 1$, lấy $k = ml_S(i) - 1$.

Nếu $mr_S(k) \geq i + 1$, thì $M_S(k) \supseteq [i, i + 1]$. Vì $[ml_S(i), mr_S(i)]$ là khoảng cực tiểu chứa $[i, i + 1]$, $M_S(k) \supseteq M_S(i)$. Lại có $k \in M_S(k)$ và $k \notin M_S(i)$, nên $M_S(k) \supsetneq M_S(i)$, mâu thuẫn với $i \in T$.

Nếu $mr_S(k) \leq i$, thì $k \leq ml_S(i) \leq mr_S(k) \leq i$. Theo bổ đề 4.1, $mr_S(k) = i$. Mặt khác, $k \leq ml_S(j) \leq mr_S(k) \leq j$, nên theo bổ đề 4.1, $mr_S(k) = j$, mâu thuẫn.

Do đó $ml_S(i) = 1$. Tương tự, $mr_S(i) = n$. Theo tính chất của T , $\forall t \in T$, không thể có $M_S(t) \subsetneq M_S(i) = [1, n]$, do đó $\forall t \in T$, $M_S(t) = [1, n]$. $\dashv$

Bổ đề 4.5. Khi các $M_S(t)$, $t \in T$, đôi một khác nhau, không tồn tại $i, j \in T$, $i \neq j$, sao cho $M_S(i) \supseteq [j, j + 1]$.

Chứng minh. Nếu tồn tại $i, j \in T$, $i \neq j$, sao cho $M_S(i) \supseteq [j, j + 1]$, thì theo định nghĩa khoảng cực tiểu, $M_S(i) \supseteq M_S(j)$. Do $M_S(i) \neq M_S(j)$, ta có $M_S(i) \supsetneq M_S(j)$, mâu thuẫn với $j \in T$. $\dashv$

Bổ đề 4.6. Khi các $M_S(t)$, $t \in T$, đôi một khác nhau, giả sử $T = \{t_1, t_2, \dots, t_k\}$, $t_0 = 0 < t_1 < t_2 < \dots < t_k < t_{k+1} = n$. Khi đó

$$M_S(t_i) = [t_{i-1} + 1, t_{i+1}].$$

Chứng minh. Theo bổ đề 4.5, $M_S(t_i) \subseteq [t_{i-1} + 1, t_{i+1}]$. Chỉ cần chứng minh

$$ml_S(t_i) = \begin{cases} 1, & i = 1, \\ t_{i-1} + 1, & i \geq 2, \end{cases}$$

phần về $mr_S(t_i)$ tương tự.

Nếu $ml_S(t_1) > 1$, thì $[1, 2]$ không bị bất kỳ $M_S(t_i)$ nào chứa, mâu thuẫn với bổ đề 4.3. Vì vậy $ml_S(t_1) = 1$.

Với $i \in [2, k]$, nếu $mr_S(t_{i-1}) < ml_S(t_i)$, thì $[ml_S(t_i) - 1, ml_S(t_i)]$ không bị bất kỳ $M_S(t_i)$ nào chứa, mâu thuẫn với bổ đề 4.3. Do đó $t_{i-1} + 1 \leq ml_S(t_i) \leq mr_S(t_{i-1}) \leq t_i$. Theo bổ đề 4.1, $ml_S(t_i) = t_{i-1} + 1$. $\dashv$

Định lý 4.3. Giả sử tập khoảng S đóng loại I, II và III. Khi $n \geq 2$, luôn có thể chia khoảng $[1, n]$ thành k khoảng con $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$, $k \geq 2$, $l_1 = 1$, $r_k = n$, $l_i \leq r_i$, $r_i + 1 = l_{i+1}$, sao cho:

- Với mọi $i \in [1, k]$, $[l_i, r_i] \in S$.
- Mọi phần tử của S hoặc bị chứa trong một khoảng con $[l_i, r_i]$, hoặc có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn liên tiếp, tức là

$$S \subseteq \bigcup_{i=1}^k \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\} \cup \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

- Tập các khoảng liên tiếp có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn thỏa một trong hai điều kiện:

$$S \supseteq \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

Khi đó gọi phép chia này là phép chia hợp.

- Hoặc S chỉ chứa các khoảng con nguyên vẹn và $[1, n]$, đồng thời $k \geq 3$:

$$S \cap \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\} = \{[l_i, r_i] \mid i \in [1, k]\} \cup \{[1, n]\}.$$

Khi đó gọi phép chia này là phép chia tách.

Chứng minh. Lấy

$$T = \{t_1, t_2, \dots, t_k\} = \{i \in [1, n-1] \mid \neg(\exists j \in [1, n-1], M_S(i) \subsetneq M_S(j))\},$$

với $t_0 = 0 < t_1 < \dots < t_k < t_{k+1} = n$. Chia $[1, n]$ thành

$$[t_0 + 1, t_1], [t_1 + 1, t_2], \dots, [t_k + 1, t_{k+1}].$$

Với khoảng con $[t_i + 1, t_{i+1}]$, theo bổ đề 4.2, $\forall j \in [t_i + 1, t_{i+1} - 1]$, $M_S(j) \subseteq [t_i + 1, t_{i+1}]$. Theo định lý 4.2, $[t_i + 1, t_{i+1}] \in S$.

Nếu các $M_S(t)$, $t \in T$, đôi một khác nhau, theo bổ đề 4.6, $M_S(t_i) = [t_{i-1} + 1, t_{i+1}]$. Theo định lý 4.2, phép chia này thỏa $S \supseteq \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}$, nên là phép chia hợp.

Ngược lại, $k \geq 2$ và theo bổ đề 4.4, $\forall t \in T, M_S(t) = [1, n]$. Theo định lý 4.2, phép chia này thỏa

$$S \cap \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\} = \{[l_i, r_i] \mid i \in [1, k]\} \cup \{[1, n]\},$$

nên là phép chia tách.

Trong cả hai trường hợp đều có $M_S(t_i) \supseteq [t_{i-1} + 1, t_{i+1}]$. Với

$$[l', r'] \in S \setminus \bigcup_{i=1}^k \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\},$$

giả sử $t_i + 1 \leq l' \leq t_{i+1} \leq t_j + 1 \leq r' \leq t_{j+1}$. Theo định lý 4.2, $[l', r'] \supseteq M_S(t_{i+1})$ và $[l', r'] \supseteq M_S(t_j)$, do đó $[l', r'] = [t_i + 1, t_{j+1}]$. Suy ra phép chia thỏa điều kiện đã nêu. $\dashv$

Định lý 4.4. Phép chia trong định lý 4.3 là duy nhất.

Chứng minh. Giả sử trên S , khoảng $[1, n]$ có hai phép chia:

$$[l_{i,1}, r_{i,1}], [l_{i,2}, r_{i,2}], \dots, [l_{i,k_i}, r_{i,k_i}] \quad (i \in [1, 2], k_i \geq 2).$$

Tìm i nhỏ nhất sao cho $[l_{1,i}, r_{1,i}] \neq [l_{2,i}, r_{2,i}]$; không mất tổng quát, giả sử $r_{1,i} < r_{2,i}$.

Nếu phép chia 1 là phép chia hợp, thì khi $i = 1$, đối với phép chia 2, khoảng $[r_{1,i} + 1, n]$ vừa không bị một khoảng con $[l_{2,j}, r_{2,j}]$ chứa, vừa không thể biểu diễn thành hợp của một số khoảng con nguyên vẹn, mâu thuẫn. Khi $i \geq 2$, đối với phép chia 2, khoảng $[1, r_{1,i}]$ cũng gây mâu thuẫn tương tự.

Nếu phép chia 1 là phép chia tách, thì khoảng $[l_{2,i}, r_{2,i}]$ vừa không bị một khoảng con $[l_{1,i}, r_{1,i}]$ chứa, vừa không phải $[1, n]$, mâu thuẫn. Vì vậy giả thiết ban đầu không đúng. $\dashv$

4.3. Cây tách–hợp

Cây tách–hợp do Liu Cheng’ao nêu ra trong trao đổi học viên WC2019 và cho thuật toán xây dựng tuyến tính; có thể xem tài liệu [3]. Định nghĩa ở đây hơi khác định nghĩa trong slide trao đổi ban đầu, nhưng bản chất là như nhau.

Định nghĩa 4.6 (cây tách–hợp). Cây tách–hợp của một tập khoảng là cây có gốc gồm ba loại nút:

- Nút lá (L, i) , biểu diễn khoảng $[i, i]$.
- Nút hợp (J, l, r) , biểu diễn khoảng $[l, r]$.
- Nút tách (C, l, r) , biểu diễn khoảng $[l, r]$.

Cây tách–hợp của tập khoảng S phải thỏa các ràng buộc sau:

- Nút gốc biểu diễn khoảng $[1, n]$.
- Mọi khoảng do nút biểu diễn đều thuộc S .
- Mỗi nút không phải lá có ít nhất hai con, và các khoảng do các con này biểu diễn tạo thành một phép chia của khoảng do nút đó biểu diễn. Riêng nút tách có ít nhất ba con.
- Với nút không phải lá (c, u, v) , $c \in \{J, C\}$, giả sử các khoảng do con của nó biểu diễn là $[l_1, r_1], [l_2, r_2], \dots, [l_k, r_k]$, $k \geq 2$, $l_1 = u$, $r_k = v$, $l_i \leq r_i$, $r_i + 1 = l_{i+1}$. Mọi khoảng trong S nằm trong $[u, v]$ hoặc bị chứa trong một khoảng con $[l_i, r_i]$, hoặc có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn:

$$S \cap \{[l, r] \mid [l, r] \subseteq [u, v]\} \subseteq \bigcup_{i=1}^k \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\} \cup \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

- Với nút hợp (J, u, v) , S chứa mọi khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn:

$$S \supseteq \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\}.$$

- Với nút tách (C, u, v) , $k \geq 3$ và trong S , những khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn chỉ gồm các khoảng con và khoảng toàn bộ:

$$S \cap \{[l_i, r_j] \mid 1 \leq i \leq j \leq k\} = \{[l_i, r_i] \mid i \in [1, k]\} \cup \{[u, v]\}.$$

Ở đây L viết tắt của leaf, J của join, và C của cut.

Định lý 4.5. Với tập khoảng S đóng loại I, II và III, cây tách–hợp của S tồn tại và duy nhất.

Chứng minh. Khi $n = 1$, cây chỉ gồm một nút lá $(L, 1)$ thỏa điều kiện, và hiển nhiên không có cây nào khác.

Giả sử kết luận đúng với $n \leq N$ ($N \geq 1$). Khi $n = N + 1 \geq 2$, chia $[1, n]$ theo định lý 4.3 thành k khoảng con $[l_1, r_1], \dots, [l_k, r_k]$. Để kiểm tra rằng với mọi $i \in [1, k]$,

$$S \cap \{[l, r] \mid [l, r] \subseteq [l_i, r_i]\},$$

sau khi tịnh tiến các phần tử, vẫn đóng loại I, II và III. Theo giả thiết quy nạp, mỗi tập con này có cây tách–hợp duy nhất.

Nếu S chứa mọi khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn, tạo nút hợp $(J, 1, n)$. Ngược lại, trong S , các khoảng có thể biểu diễn thành hợp của một số khoảng con nguyên vẹn chỉ là $[l_1, r_1], \dots, [l_k, r_k]$ và $[1, n]$; tạo nút tách $(C, 1, n)$. Gắn gốc của k cây con sau khi tịnh tiến về vị trí ban đầu làm k con của nút mới. Để kiểm tra cây thu được thỏa điều kiện, nên cây tách–hợp của S tồn tại.

Khi $n = N + 1 \geq 2$, các khoảng do con của gốc biểu diễn tạo thành phép chia trong định lý 4.3. Theo định lý 4.4, phép chia này duy nhất, nên với mọi cây tách–hợp của S , kiểu của gốc và các khoảng do con của gốc biểu diễn đều giống nhau. Theo giả thiết quy nạp, mỗi cây con cũng duy nhất, do đó cây tách–hợp của S duy nhất. $\dashv$

Định lý 4.6. Với tập khoảng S , cây tách–hợp của S tồn tại khi và chỉ khi S đóng loại I, II và III.

Chứng minh. Chiều đủ đã được định lý 4.5 cho.

Chiều cần: giả sử cây tách–hợp của S tồn tại. Vì trong cây chắc chắn có n nút lá (L, i) , $i \in [1, n]$, và nút gốc biểu diễn $[1, n]$, ta có $[1, n] \in S$ và $\forall i \in [1, n]$, $\{i\} \in S$, nên S chính quy.

Với hai khoảng $[l_1, r_1]$, $[l_2, r_2]$ có giao không rỗng, gọi u là nút sâu nhất có khoảng biểu diễn chứa $[l_1, r_1]$, và v là nút sâu nhất có khoảng biểu diễn chứa $[l_2, r_2]$. Lấy $i \in [l_1, r_1] \cap [l_2, r_2]$. Vì u và v đều là tổ tiên của (L, i) , chúng có quan hệ tổ tiên–hậu duệ; không mất tổng quát, nếu $u \neq v$ thì u là tổ tiên của v .

Nếu $u = v$, xét theo loại của u . Nếu u là lá hoặc nút tách, thì $[l_1, r_1]$ và $[l_2, r_2]$ đều chính là khoảng do u biểu diễn, nên hợp và giao của chúng đều thuộc S . Nếu u là nút hợp, thì $[l_1, r_1] \cap [l_2, r_2]$ và $[l_1, r_1] \cup [l_2, r_2]$ đều là hợp của một số khoảng con trong phép chia của u , nên thuộc S .

Nếu u là tổ tiên của v , thì $[l_1, r_1] \cup [l_2, r_2] = [l_1, r_1] \in S$. Vì vậy S đóng loại I và II.

Nếu $[l_1, r_1]$ và $[l_2, r_2]$ không chứa nhau, thì $u = v$ và u là nút hợp. Khi đó $[l_1, r_1] \setminus [l_2, r_2]$ là hợp của một số khoảng con trong phép chia của khoảng do u biểu diễn, nên theo tính chất của nút hợp, nó thuộc S . Do đó S đóng loại III. $\dashv$

Ví dụ, với

$$S = \{[1, 9], [1, 2], [3, 6], [5, 6], [7, 9], [7, 8], [8, 9]\} \cup \{[i, i] \mid i \in [1, 9]\},$$

cây tách–hợp có gốc là nút tách $C, 1, 9$; các nút trong cây gồm các nút hợp $J, 1, 2, J, 5, 6, J, 7, 9$, nút tách $C, 3, 6$, và các lá $L, 1, \dots, L, 9$.

4.4. Xây dựng cây tách–hợp

Trong bài toán thực tế, dãy khoảng cực tiểu của tập khoảng thường không dễ tìm nhanh, vì vậy ta đưa vào dãy khoảng cực tiểu giả.

Định nghĩa 4.7 (dãy khoảng cực tiểu giả). Dãy khoảng $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của tập khoảng đóng loại I và II S khi và chỉ khi

$$\forall i \in [1, n-1], \quad [l_i, r_i] \supseteq [i, i+1],$$

và

$$\forall 1 \leq u \leq v \leq n, \quad [u, v] \in S \iff (\forall i \in [u, v-1], [l_i, r_i] \subseteq [u, v]).$$

Nói cách khác, dãy khoảng cực tiểu giả có tính chất của khoảng cực tiểu được mô tả trong định lý 4.2.

4.4.1. Thuật toán thô. Cho dãy khoảng cực tiểu giả của tập khoảng đóng loại I, II và III S , có thể dùng phương pháp tham lam gộp dần để xây dựng cây tách–hợp của S .

Thuật toán 1. Xây dựng thô cây tách–hợp từ dãy khoảng cực tiểu giả.

Yêu cầu: $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của S . Kết quả: ngăn xếp s cuối cùng chứa nút gốc của cây tách–hợp.

1. $s \leftarrow$ ngăn xếp rỗng.
2. Với $i = 1$ đến n :
3. $u \leftarrow \text{Node}(L, i, i, \emptyset)$, lần lượt biểu thị kiểu, trái, phải và danh sách con.
4. $\text{sufmin} \leftarrow i, \text{sufmax} \leftarrow i, m \leftarrow |s|$.
5. Với $j = m$ giảm đến 1:
6. $\text{sufmin} \leftarrow \min\{\text{sufmin}, l_{s_j.\text{right}}\}$.
7. $\text{sufmax} \leftarrow \max\{\text{sufmax}, r_{s_j.\text{right}}\}$.
8. Nếu $\text{sufmin} \geq s_j.\text{left}$ và $\text{sufmax} \leq i$:
9. Nếu $j = |s|$:
10. Nếu $s_j.\text{type} = J$ và $\text{sufmin} \geq s_j.\text{child}_{|s_j.\text{child}|}.\text{left}$, thêm u vào con của s_j , đặt $s_j.\text{right} \leftarrow i$, $u \leftarrow s_j$.
11. Ngược lại, đặt $u \leftarrow \text{Node}(J, s_j.\text{left}, i, (s_j, u))$.
12. Ngược lại, đặt $u \leftarrow \text{Node}(C, s_j.\text{left}, i, (s_j, s_{j+1}, \dots, s_{|s|}, u))$.
13. Trong khi $|s| \geq j$, lấy phần tử khỏi s .
14. Đưa u vào s .

Có thể thấy các nút mới mà thuật toán tạo ra luôn biểu diễn các khoảng thuộc S . Với một nút không phải lá trong cây tách–hợp của S , thuật toán không gộp các nút thuộc các cây con khác nhau, đồng thời thiết lập đúng nút hợp và nút tách. Vì vậy thuật toán luôn cho kết quả đúng.

4.4.2. Thuật toán tuyến tính. Do tổng số nút của cây tách–hợp là $O(n)$, nút thất của thuật toán thô là tìm nút mới cần tạo.

Trong thuật toán thô, nếu $sufmax > i$, thì với i hiện tại không thể tạo nút mới nữa, nên có thể thoát ngay khỏi vòng lặp trong. Nếu $l_{s_j.right} < s_j.left$, thì s_j về sau không thể làm đầu trái của một nút mới nào nữa, nên không cần xét lại. Dùng hai ý tưởng tối ưu này, ta thu được thuật toán tuyến tính.

Thuật toán 2. Xây dựng tuyến tính cây tách–hợp từ dãy khoảng cực tiểu giả.

Yêu cầu: $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của S . Kết quả: ngăn xếp s cuối cùng chứa nút gốc của cây tách–hợp.

1. Đặt $l_n \leftarrow 1$, $s \leftarrow$ ngăn xếp rỗng, $t \leftarrow$ ngăn xếp rỗng.
2. Với $i = 1$ đến n :
3. $u \leftarrow \text{Node}(L, i, i, \emptyset)$.
4. Trong khi t không rỗng và $r_{t.top()} \leq i$:
5. $j \leftarrow |s|$.
6. Trong khi $s_j.left > t.top()$, đặt $j \leftarrow j - 1$.
7. Nếu $j = |s|$:
8. Nếu $s_j.type = J$ và $l_{s_j.right} \geq s_j.child_{|s_j.child|-1}.left$, thêm u vào con của s_j , đặt $s_j.right \leftarrow i$, $u \leftarrow s_j$.
9. Ngược lại, đặt $u \leftarrow \text{Node}(J, s_j.left, i, (s_j, u))$.
10. Nếu không, đặt $u \leftarrow \text{Node}(C, s_j.left, i, (s_j, s_{j+1}, \dots, s_{|s|}, u))$.
11. Trong khi $|s| \geq j$, lấy phần tử khỏi s .
12. Lấy phần tử khỏi t .
13. Đưa u vào s .
14. Đưa $u.left$ vào t .
15. Trong khi $t.top() > l_i$, lấy phần tử khỏi t .
16. Đặt $r_{t.top()} \leftarrow \max\{r_{t.top()}, r_i\}$.

Có thể kiểm tra thuật toán tuyến tính trên tương đương với thuật toán thô, nên nó luôn cho kết quả đúng.

4.5. Tổng thông tin tại đầu trái ứng với đầu phải

Cho tập khoảng S và n thông tin $a_1, a_2, \dots, a_n$. Phép gộp thông tin thỏa tính chất nửa nhóm, tức là đóng và kết hợp; ký hiệu phép gộp là $\times$. Cần với mỗi $r \in [1, n]$ tính

$$\prod_{l \in [1, r], [l, r] \in S} a_l.$$

Với tập khoảng đóng loại I, II và III, có thể dựng cây tách–hợp rồi giải bằng một lần DFS.

Với tập khoảng đóng loại I và II, nếu phép gộp thông tin thỏa tính chất nhóm, tức ngoài tính chất nửa nhóm còn có đơn vị và mỗi phần tử có nghịch đảo, thì có thuật toán tuyến tính. Ký hiệu đơn vị là 1, nghịch đảo của a là a^{-1} .

Thuật toán 3. Tuyến tính để tìm tích thông tin tại đầu trái ứng với đầu phải.

Yêu cầu: $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$ là dãy khoảng cực tiểu giả của S , và phép nhân của các a_i thỏa tính chất nhóm. Kết quả:

$$b_i = \prod_{j \in [1, i], [i, j] \in S} a_j.$$

1. Với $i = 1$ đến $n - 1$, đặt $c_i \leftarrow 0$.
2. $t \leftarrow$ ngăn xếp rỗng.
3. Với $i = n - 1$ giảm đến 1:
4. Đưa i vào t .
5. Trong khi $t.top() < r_i - 1$:
6. $c_{t.top()} \leftarrow i$.
7. Lấy phần tử khỏi t .
8. $s \leftarrow$ ngăn xếp rỗng, đưa 0 vào s .
9. $p_0 \leftarrow 1, b_1 \leftarrow a_1$.
10. Với $i = 1$ đến $n - 1$:
11. $p_i \leftarrow p_{s.top()} \times a_i$.
12. Đưa i vào s .
13. Trong khi $s.top() > l_i$:
14. $\text{Erase}(s.top())$.
15. Lấy phần tử khỏi s .
16. $v \leftarrow \text{Query}(c_i)$.
17. $b_{i+1} \leftarrow p_v^{-1} \times p_{s.top()} \times a_{i+1}$.

Giải thích. Trong thuật toán, c_i là k lớn nhất trong $[1, i]$ sao cho $r_k > i + 1$; nếu không tồn tại thì $c_i = 0$. Ràng buộc trên r trong dãy khoảng cực tiểu giả được chuyển thành $j > c_i$, còn ràng buộc trên l được chuyển thành thao tác lấy khỏi đỉnh ngăn xếp s . Thuật toán cần một cấu trúc dữ liệu:

- Duy trì một xâu 01 độ dài $n + 1$, ban đầu toàn 1.
- $\text{Erase}(x)$: đổi s_x thành 0.
- $\text{Query}(x)$: truy vấn y lớn nhất sao cho $y \leq x$ và $s_y = 1$.

Cài đặt cấu trúc dữ liệu. Trong mô hình Word RAM, có thể dùng DSU nén bit để cài đặt cấu trúc dữ liệu này. Đại khái, chia xâu s thành các khối w bit, xử lý trong khối bằng phép toán bit trong $O(1)$, còn giữa các khối dùng DSU có nén đường đi và gộp theo kích thước. Theo [2], độ phức tạp là $O(n\alpha(n, n/w)) = O(n)$.

Thuật toán trên, cũng như cách giải bằng cây tách–hợp, đều là trực tuyến: có thể lần lượt tính đáp án cho $r = 1, 2, \dots, n$. Khi tính đáp án cho $r = r_0$, không cần biết $a_{r_0+1}, \dots, a_n$.

5. Phân tích thêm về đoạn liên tiếp

Tập liên thông của dây chuyền, tức hoán vị, là chính quy và đóng loại I, II, III. Tập liên thông của cây là chính quy và đóng loại I, II. Tập liên thông của đồ thị là chính quy và đóng loại I.

Vì giao của một số họ tập cùng thỏa một trong các tính chất chính quy, đóng loại I, đóng loại II, đóng loại III vẫn thỏa tính chất đó, ta có thể trực tiếp suy ra tính chất của một số tổ hợp cấu trúc.

5.1. Hai dây chuyền

5.1.1. Tính chất. Đánh số lại các phần tử trên một dây chuyền theo thứ tự $1, 2, \dots, n$, ta thu được rằng đoạn liên tiếp của hai dây chuyền tương đương với đoạn liên tiếp của hoán vị. Tương tự, mọi tổ hợp chứa dây chuyền đều có thể chuyển đổi theo cách này.

Tập đoạn liên tiếp của hai dây chuyền là chính quy và đóng loại I, II, III, nghĩa là có thể dùng cây tách–hợp để mô tả đoạn liên tiếp của hoán vị.

Một dãy khoảng cực tiểu giả của tập đoạn liên tiếp của hoán vị p là

$$\left(\left[\min_{j \in A_i} p_j^{-1}, \max_{j \in A_i} p_j^{-1} \right] \right)_{i \in [1, n-1]},$$

trong đó

$$A_i = [\min\{p_i, p_{i+1}\}, \max\{p_i, p_{i+1}\}].$$

5.1.2. Độ phức tạp. Định lý 5.1. Với một cây tách–hợp, nếu mọi nút tách của nó đều có ít nhất bốn con, thì tồn tại một hoán vị p sao cho cây tách–hợp của tập đoạn liên tiếp của p chính là cây đã cho.

Định nghĩa $\text{combine}(p, q_1, q_2, \dots, q_{|p|})$ là hoán vị duy nhất thu được bằng cách cộng cùng một số vào toàn bộ mỗi q_i rồi ghép theo thứ tự, sao cho thứ tự tương đối giữa các q_i khớp với p . Ví dụ:

$$\text{combine}((1, 3, 2), (1, 2), (2, 1), (1)) = (1, 2, 5, 4, 3).$$

Định nghĩa

$$\text{flip}(p) = (|p| + 1 - p_i)_{i \in [1, |p|]}.$$

Ví dụ $\text{flip}((1, 4, 2, 3)) = (4, 1, 3, 2)$.

Với một cây tách–hợp thỏa điều kiện, có thể xây dựng p như sau:

- Nếu gốc là lá, đặt $p = (1)$.
- Ngược lại, giả sử các cây con được xây dựng thành $p_1, \dots, p_k$.
- Nếu gốc là nút hợp, đặt

$$p = \text{combine}((1, \dots, k), \text{flip}(p_1), \dots, \text{flip}(p_k)).$$

- Nếu gốc là nút tách, đặt

$$p = \text{combine}(q, \text{flip}(p_1), \dots, \text{flip}(p_k)),$$

trong đó

$$q = \begin{cases} (2, 4, \dots, k, 1, 3, \dots, k-1), & 2 \mid k, \\ (2, 4, \dots, k-1, 1, k, 3, \dots, k-2), & 2 \nmid k. \end{cases}$$

5.2. Nhiều dây chuyền hơn

5.2.1. Tính chất. Định nghĩa 5.1 (đoạn liên tiếp của nhiều hoán vị). Với dãy gồm m hoán vị $p = (p_1, \dots, p_m)$, nếu khoảng $[l, r]$ là đoạn liên tiếp của mọi p_i , thì gọi $[l, r]$ là đoạn liên tiếp của p .

Đoạn liên tiếp của k dây chuyền ($k \geq 3$) tương đương với đoạn liên tiếp của $k-1$ hoán vị.

Tập đoạn liên tiếp của k dây chuyền là chính quy và đóng loại I, II, III, nghĩa là vẫn có thể dùng cây tách-hợp để mô tả đoạn liên tiếp của nhiều hoán vị.

Một dãy khoảng cực tiểu giả của tập đoạn liên tiếp của dãy hoán vị p là dãy thu được bằng cách lấy hợp theo từng vị trí của các dãy khoảng cực tiểu giả của từng hoán vị p_i .

5.2.2. Độ phức tạp. Định lý 5.2. Với mọi cây tách-hợp, tồn tại hai hoán vị p_1, p_2 sao cho cây tách-hợp của tập đoạn liên tiếp của (p_1, p_2) chính là cây đó.

Vì mọi tập khoảng đóng loại I, II, III đều tương ứng với một cây tách-hợp duy nhất, định lý này thực chất chỉ ra rằng tập đoạn liên tiếp của hai hoán vị có thể là mọi tập khoảng đóng loại I, II, III.

Với một cây tách-hợp thỏa điều kiện, có thể xây dựng cặp hoán vị (p, q) như sau:

- Nếu gốc là lá, đặt $(p, q) = ((1), (1))$.
- Ngược lại, giả sử các cây con được xây dựng thành các cặp hoán vị $(p_1, q_1), \dots, (p_k, q_k)$.
- Nếu gốc là nút hợp, đặt

$$p = \text{combine}((1, \dots, k), \text{flip}(p_1), \dots, \text{flip}(p_k)),$$

$$q = \text{combine}((1, \dots, k), \text{flip}(q_1), \dots, \text{flip}(q_k)).$$

- Nếu gốc là nút tách, đặt

$$p = \text{combine}(p', \text{flip}(p_1), \dots, \text{flip}(p_k)),$$

$$q = \text{combine}(q', \text{flip}(q_1), \dots, \text{flip}(q_k)),$$

trong đó

$$p' = \begin{cases} (2, 3, 1), & k = 3, \\ (2, 4, \dots, k, 1, 3, \dots, k-1), & k \geq 4, 2 \mid k, \\ (2, 4, \dots, k-1, 1, k, 3, \dots, k-2), & k \geq 4, 2 \nmid k, \end{cases}$$

và

$$q' = \begin{cases} (3, 1, 2), & k = 3, \\ (2, 4, \dots, k, 1, 3, \dots, k-1), & k \geq 4, 2 \mid k, \\ (2, 4, \dots, k-1, 1, k, 3, \dots, k-2), & k \geq 4, 2 \nmid k. \end{cases}$$

Chẳng hạn, với cây tách-hợp ở ví dụ trước, có thể xây dựng hai hoán vị

$$(5, 4, 9, 8, 6, 7, 2, 1, 3), \quad (9, 8, 4, 3, 2, 1, 5, 7, 6).$$

5.3. Dây chuyền và cây

5.3.1. Tính chất. **Định nghĩa 5.2** (đoạn liên tiếp của cây). Với cây $T = (V, E)$, $V = [1, n]$, nếu khoảng $[l, r]$ thỏa đồ thị con cảm sinh của T trên tập đỉnh $[l, r]$ là đồ thị liên thông, thì gọi $[l, r]$ là đoạn liên tiếp của T .

Tổ hợp “dây chuyền, cây” tương đương với đoạn liên tiếp của cây.

Tập đoạn liên tiếp của tổ hợp “dây chuyền, cây” là chính quy và đóng loại I, II, nghĩa là có thể dùng $n - 1$ khoảng cực tiểu để mô tả đoạn liên tiếp của cây.

Một dãy khoảng cực tiểu giả của tập đoạn liên tiếp của cây là

$$([\min \text{path}(i, i + 1), \max \text{path}(i, i + 1)])_{i \in [1, n-1]},$$

trong đó $\text{path}(i, i + 1)$ biểu thị tập đỉnh trên đường đi từ i đến $i + 1$ trong cây.

5.3.2. Độ phức tạp. Tập đoạn liên tiếp của cây không đóng loại III.

Xét cây $T = (V, E)$, $V = \{1, 2, 3, 4\}$, $E = \{(1, 3), (2, 3), (3, 4)\}$. Các khoảng $[1, 3]$ và $[3, 4]$ đều là đoạn liên tiếp của nó, nhưng

$$[1, 3] \setminus [3, 4] = [1, 2]$$

không phải đoạn liên tiếp của nó.

5.4. Dây chuyền và nhiều cây

5.4.1. Tính chất. **Định nghĩa 5.3** (đoạn liên tiếp của nhiều cây). Với dãy gồm m cây $T = (T_1, \dots, T_m)$, $V(T_i) = [1, n]$, nếu khoảng $[l, r]$ là đoạn liên tiếp của mọi T_i , thì gọi $[l, r]$ là đoạn liên tiếp của T .

Tổ hợp “dây chuyền, nhiều cây” tương đương với đoạn liên tiếp của nhiều cây.

Tập đoạn liên tiếp của tổ hợp này là chính quy và đóng loại I, II, nghĩa là có thể dùng $n - 1$ khoảng cực tiểu để mô tả đoạn liên tiếp của nhiều cây.

Một dãy khoảng cực tiểu giả của tập đoạn liên tiếp của dãy cây T là dãy thu được bằng cách lấy hợp theo từng vị trí của các dãy khoảng cực tiểu giả của từng cây T_i .

5.4.2. Độ phức tạp. Với dãy gồm $n - 1$ khoảng $A = (A_1, \dots, A_{n-1})$, $A_i = [u_i, v_i]$, nếu với mọi $i \in [1, n - 1]$ đều có $A_i \supseteq [i, i + 1]$ và

$$\forall j \in [u_i, v_i - 1], \quad A_j \subseteq A_i,$$

thì gọi A là hợp lệ.

Từ định nghĩa khoảng cực tiểu và bổ đề 4.2, dãy khoảng cực tiểu M_S của tập khoảng đóng loại I và II là hợp lệ.

Định lý 5.3. Với mọi dãy hợp lệ A gồm $n - 1$ khoảng, tồn tại hai cây T_1, T_2 sao cho dãy khoảng cực tiểu của tập đoạn liên tiếp của (T_1, T_2) chính là A .

Vì mọi tập khoảng đóng loại I và II đều tương ứng với một dãy khoảng cực tiểu, và dãy khoảng cực tiểu luôn hợp lệ, định lý này thực chất chỉ ra rằng tập đoạn liên tiếp của hai cây có thể là mọi tập khoảng đóng loại I và II.

Với một dãy khoảng cực tiểu $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$, đặt

$$V = [1, n], \quad E_1 = \{(l_i, i + 1) \mid i \in [1, n - 1]\},$$

$$E_2 = \{(r_i, i) \mid i \in [1, n - 1]\}, \quad T_1 = (V, E_1), \quad T_2 = (V, E_2).$$

Có thể kiểm tra dãy khoảng cực tiểu của tập đoạn liên tiếp của (T_1, T_2) chính là $([l_1, r_1], \dots, [l_{n-1}, r_{n-1}])$.

5.5. Dây chuyền và đồ thị

5.5.1. Tính chất. **Định nghĩa 5.4** (đoạn liên tiếp của đồ thị). Với đồ thị $G = (V, E)$, $V = [1, n]$, nếu khoảng $[l, r]$ thỏa đồ thị con cảm sinh của G trên tập đỉnh $[l, r]$ là đồ thị liên thông, thì gọi $[l, r]$ là đoạn liên tiếp của G .

Tổ hợp “dây chuyền, đồ thị” tương đương với đoạn liên tiếp của đồ thị.

Tập đoạn liên tiếp của tổ hợp này là chính quy và đóng loại I.

5.5.2. Độ phức tạp. Tập đoạn liên tiếp của đồ thị không đóng loại II.

Xét đồ thị $G = (V, E)$, $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (2, 4), (4, 3), (3, 1)\}$. Các khoảng $[1, 3]$ và $[2, 4]$ đều là đoạn liên tiếp của nó, nhưng

$$[1, 3] \cap [2, 4] = [2, 3]$$

không phải đoạn liên tiếp của nó.

5.6. Hai cây

5.6.1. Tính chất. Tập đoạn liên tiếp của hai cây là chính quy và đóng loại I, II. Nhưng vì không có dây chuyền, ta không thể đánh số lại các phần tử sao cho đoạn liên tiếp trở thành khoảng.

5.6.2. Độ phức tạp. Đếm đoạn liên tiếp của hai cây là #P-complete.

Theo [1], đếm số phản dây chuyền trên một tập có thứ tự bộ phận cho trước là #P-complete. Do đó chỉ cần quy giảm trong thời gian đa thức bài toán đếm phản dây chuyền về bài toán đếm đoạn liên tiếp của hai cây, ta chứng minh được đếm đoạn liên tiếp của hai cây là #P-complete.

Giả sử tập có thứ tự bộ phận cho trước là $(S, \leq)$. Để tiện, đặt $S = \{1, 2, \dots, n\}$. Đặt

$$\{x_{i,1}, x_{i,2}, \dots, x_{i,k_i}\} = \{j \in S \mid i \leq j, i \neq j\}.$$

Xây dựng tập đỉnh

$$V = \{a\} \cup \{b_i \mid i \in S\} \cup \{c_i \mid i \in S\} \cup \{d_i \mid i \in S\} \cup \{e_{i,j} \mid i, j \in S, i \neq j, i \leq j\}.$$

Đặt

$$(h_{i,1}, h_{i,2}, \dots, h_{i,k_i+4}) = (a, b_i, d_i, e_{i,x_{i,1}}, e_{i,x_{i,2}}, \dots, e_{i,x_{i,k_i}}, c_i).$$

Xây dựng tập cạnh

$$E_1 = \bigcup_{i \in S} \{(h_{i,j}, h_{i,j+1}) \mid j \in [1, k_i + 3]\},$$
$$E_2 = \bigcup_{i \in S} \{(a, d_i), (a, c_i), (c_i, b_i)\} \cup \bigcup_{i, j \in S, i \neq j, i \leq j} \{(e_{i,j}, b_j)\}.$$

Số đoạn liên tiếp của hai cây $T_1 = (V, E_1)$, $T_2 = (V, E_2)$ trừ đi $|V|$ chính là số phản dây chuyền của $(S, \leq)$.

Nguyên nhân là mọi đoạn liên tiếp không tầm thường, tức đoạn có ít nhất hai phần tử, tương ứng một-một với phản dây chuyền.

Dạng của đoạn liên tiếp. Mọi đoạn liên tiếp không tầm thường đều có thể viết dưới dạng

$$\{a\} \cup \{b_i \mid i \in A\} \cup \{c_i \mid i \in A\} \cup \{d_i \mid i \in A\} \cup \{e_{i,j} \mid i \in A, j \in S, i \neq j, i \leq j\},$$

trong đó $\emptyset \neq A \subseteq S$.

Từ đoạn liên tiếp sang phản dây chuyền. Đoạn liên tiếp không tầm thường ở dạng trên tương ứng với phản dây chuyền

$$\{i \mid i \in A, \forall j \leq i, j \notin A\}.$$

Từ phản dây chuyền sang đoạn liên tiếp. Phản dây chuyền A tương ứng với đoạn liên tiếp không tầm thường

$$\{a\} \cup \{b_i \mid i \in B\} \cup \{c_i \mid i \in B\} \cup \{d_i \mid i \in B\} \cup \{e_{i,j} \mid i \in B, j \in S, i \neq j, i \leq j\},$$

trong đó

$$B = \{i \mid \exists j \in A, j \leq i\}.$$

Quá trình trên quy giảm bài toán đếm phản dây chuyền của tập có thứ tự bộ phận gồm n phần tử về bài toán đếm đoạn liên tiếp của hai cây có $O(n^2)$ đỉnh. Vì vậy đếm đoạn liên tiếp của hai cây là #P-complete; điều này có nghĩa là nếu tồn tại thuật toán đa thức cho bài toán này thì $P = NP$.

6. Tổng kết

Độ phức tạp hoặc độ khó của các bài toán và cấu trúc trong bài viết có thể biểu diễn xấp xỉ như sau:

hai dây chuyền đơn giản	nhiều dây chuyền	dây chuyền, cây	dây chuyền, đồ thị	hai cây khó phức tạp
đóng I, II, III	đóng I, II, III	đóng I, II	đã biết thuật toán đa thức	

Hai dây chuyền và nhiều dây chuyền được mô tả bằng cây tách-hợp; dây chuyền với cây hoặc nhiều cây được mô tả bằng dãy khoảng cực tiểu.

Bài viết chỉ ra bản chất của một lớp bài toán đoạn liên tiếp: giao của các tập con “liên thông” trên nhiều cấu trúc. Bài viết dùng dãy khoảng cực tiểu để mô tả tập khoảng S đóng loại I và II, đồng thời đưa ra thuật toán tuyến tính để, với một đầu phải r , tính tổng thông tin tại mọi đầu trái l thỏa $[l, r] \in S$. Tuy nhiên thuật toán này cần thông tin được duy trì có nghịch đảo, nên còn khá hạn chế. Do năng lực có hạn, tác giả chưa tìm được thuật toán cho thông tin nửa nhóm hoặc chứng minh rằng thuật toán như vậy không tồn tại.

Ngoài ra, bài viết cũng đưa ra điều kiện cần và đủ để tập khoảng S có cây tách-hợp, tức là S đóng loại I, II và III. Cuối cùng, bài viết dùng thuật toán trên tập khoảng để giải các bài toán đoạn liên tiếp, đồng thời chứng minh độ phức tạp hoặc độ khó của một số bài toán đoạn liên tiếp, chẳng hạn đếm số đoạn liên tiếp của hai cây là #P-complete.

Hy vọng bài viết giúp người đọc hiểu sâu hơn về đoạn liên tiếp và tiếp tục nghiên cứu thêm.

7. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yang Yaoliang, Peng Sijin và Zhang Yibin đã chỉ đạo. Xin cảm ơn cha mẹ đã nuôi dưỡng và giáo dục tác giả. Xin cảm ơn nhà trường đã bồi dưỡng, cảm ơn các thầy Fu Shuibao, Ying Ping'an, Yu Ting, Lin Naijie, Dong Yi đã giảng dạy, và cảm ơn các bạn học đã giúp đỡ. Xin cảm ơn các bạn học tại Trường Trung học Trần Hải đã đọc kiểm bản thảo. Xin cảm ơn Liu Cheng'ao đã trao đổi với tác giả và đem lại cảm hứng cho bài viết.

Tài liệu tham khảo

1. Scott Provan and Michael O. Ball. The complexity of counting cuts and of computing the probability that a graph is connected. *SIAM Journal on Computing*, 12, 1983.
2. Jan van Leeuwen and Robert E. Tarjan. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31:245–281, 1984.
3. Liu Cheng’ao. Cấu trúc dữ liệu đơn giản cho đoạn liên tiếp. Trao đổi học viên WC2019, 2019.

Bài 2. Các phương pháp chứng minh tính lỗi trong đề OI

Gou Jingyu, Trường Trung học Nam Khai, Trùng Khánh

Tóm tắt

Tính lỗi thường xuất hiện trong các bài toán Olympic Tin học. Khi một bài toán có tính lỗi, ta có thể dùng tìm kiếm tam phân, tối ưu hóa quy hoạch động bằng bất đẳng thức tứ giác, WQS, hay các kỹ thuật liên quan tới bao lỗi. Tuy nhiên, nhiều tài liệu chỉ tập trung vào cách dùng tính lỗi sau khi đã có nó, còn việc chứng minh tính lỗi lại bị bỏ qua hoặc chỉ được trình bày rất sơ lược. Bài viết này tổng kết một số cách chứng minh tính lỗi thường gặp trong đề OI: quy về mô hình thuật toán quen thuộc, chứng minh bằng đại số, và chứng minh dựa trên bản chất hình học của bài toán.

1. Lời nói đầu

Trong lời giải OI, tính lỗi thường đóng vai trò là chiếc cầu nối giữa nhận xét toán học và thuật toán hiệu quả. Một khi chứng minh được hàm mục tiêu hoặc dãy trạng thái có tính lỗi, ta có thể suy ra tính đơn điệu của điểm quyết định, tính đơn đỉnh, hoặc tính chất của sai phân, từ đó giảm độ phức tạp đáng kể.

Khó khăn là nhiều tính chất này không nằm ngay trên bề mặt đề bài. Người giải thường quan sát bằng bảng giá trị hoặc bằng trực giác thuật toán, nhưng một lời giải hoàn chỉnh vẫn cần chứng minh chặt chẽ. Vì vậy trọng tâm của bài viết là “chứng minh tính lỗi”, không phải chỉ là “sử dụng tính lỗi”.

2. Khái niệm cơ bản

Bài viết dùng cùng quy ước với một số tài liệu đội tuyển gần đây: phân biệt *lỗi trên* và *lỗi dưới*. Với hàm f trên miền thực, f là lỗi trên nếu

$$f\left(\frac{a+b}{2}\right) \geq \frac{f(a)+f(b)}{2},$$

và là lỗi dưới nếu dấu bất đẳng thức đảo chiều. Trên dãy số, a_i là lỗi trên nếu

$$2a_i \geq a_{i-1} + a_{i+1},$$

và là lỗi dưới nếu

$$2a_i \leq a_{i-1} + a_{i+1}.$$

Nói theo sai phân bậc nhất, dãy lỗi dưới có sai phân không giảm, còn dãy lỗi trên có sai phân không tăng. Nói theo sai phân bậc hai, dấu của sai phân bậc hai quyết định chiều lỗi.

Khi chứng minh trong bài toán rời rạc, cách kiểm tra bằng sai phân thường tiện hơn định nghĩa trung điểm. Với bài toán liên tục hoặc hình học, định nghĩa trung điểm lại thường tự nhiên hơn.

3. Quy về mô hình thuật toán quen thuộc

Cách chứng minh đầu tiên là nhận ra bài toán đang ẩn một mô hình đã biết có tính lời. Hai mô hình tiêu biểu là luồng chi phí và bất đẳng thức tứ giác.

3.1. Luồng chi phí

Trong mạng luồng với chi phí cạnh tuyến tính, nếu xét chi phí nhỏ nhất khi đẩy đúng k đơn vị luồng, hàm theo k là lồi dưới. Tương tự, chi phí lớn nhất theo lượng luồng là lồi trên. Có thể hiểu điều này qua đường đi tăng luồng trong đồ thị dư: các chi phí biên được chọn theo thứ tự không giảm.

Ví dụ 3.1 (House Deconstruction, dạng rút gọn). Trên một vòng tròn độ dài x có n người và m căn nhà, $n < m$. Cần gán mỗi người vào một căn nhà sao cho tổng khoảng cách trên vòng tròn là nhỏ nhất.

Khi $n = m$, có thể cắt vòng thành đường thẳng sau khi cố định số người đi qua cạnh cắt. Gọi c_i là tổng chênh lệch tiền tố giữa số người và số nhà, khi có k người đi qua cạnh cắt thì chi phí có dạng

$$h(k) = \sum_i |k + c_i|.$$

Đây là một hàm lồi dưới, nên có thể tìm cực tiểu bằng tam phân hoặc bằng sai phân.

Với $n < m$, bài toán có thể mô hình hóa bằng luồng chi phí nhỏ nhất. Nếu cố định lượng luồng k đi qua cạnh từ x về 1, chi phí tối ưu $g(k)$ là lồi dưới. Lập luận là đưa ràng buộc cận dưới/cận trên lên cạnh đó, rồi xét thay đổi một đơn vị luồng trong mạng dư. Chi phí biên của các lần tăng luồng không giảm, do đó g lồi dưới.

Bài toán còn có thể giải bằng quy hoạch động trên vị trí. Đặt $dp(i, j)$ là chi phí tối ưu sau khi quét tới vị trí i và còn chênh lệch j giữa số người và số nhà đã xử lý. Khi chuyển qua một người, một căn nhà hoặc vị trí trống, trạng thái theo j vẫn giữ tính lồi dưới. Vì vậy có thể duy trì mảng sai phân của hàm lồi thay vì tính từng giá trị độc lập, đạt độ phức tạp $O(m \log n)$ và dùng $O(n + m)$ bộ nhớ.

3.2. Bất đẳng thức tứ giác

Một hàm chi phí đoạn $w(l, r)$ thỏa bất đẳng thức tứ giác nếu với $a \leq b \leq c \leq d$ có

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c).$$

Khi w có tính chất này, nhiều quy hoạch động chia đoạn sẽ có điểm quyết định đơn điệu.

Ví dụ 3.2 (chia đoạn một tầng). Với

$$f(i) = \min_{1 \leq j \leq i} \{f(j-1) + w(j, i)\},$$

nếu w thỏa bất đẳng thức tứ giác thì điểm quyết định tối ưu của $f(i)$ không giảm theo i . Đây là cơ sở cho tối ưu hóa chia để trị hoặc tối ưu hóa bằng hàng đợi tùy dạng bài.

Ví dụ 3.3 (chia thành m đoạn). Với

$$f(i, k) = \min_{1 \leq j \leq i} \{f(j-1, k-1) + w(j, i)\},$$

nếu w thỏa bất đẳng thức tứ giác thì giá trị tối ưu theo số đoạn k là lồi dưới. Tính chất này là nền tảng cho WQS: thêm phạt tuyến tính cho mỗi đoạn, tìm số đoạn tối ưu bằng nhị phân trên hệ số phạt, rồi khôi phục đáp án mong muốn.

Ví dụ 3.4 (Bar Cover). Cho dãy độ dài n . Mỗi thẻ che một đoạn liên tiếp độ dài k , các thẻ không được chồng nhau. Cần tính tổng lớn nhất có thể che bởi đúng i thẻ, với mọi i .

Gọi đáp án theo số thẻ là $f(i)$. Có thể chứng minh $f(i)$ là lồi trên. Một cách chứng minh tổ hợp là lấy phương án tối ưu của $i - 1$ thẻ và của $i + 1$ thẻ, trộn các điểm bắt đầu đoạn rồi tách theo vị trí chắn lẻ. Hai tập thu được đều là phương án hợp lệ với i thẻ, nên tổng của chúng không vượt quá $2f(i)$. Do đó

$$f(i - 1) + f(i + 1) \leq 2f(i).$$

Cũng có thể chứng minh bằng bất đẳng thức tứ giác. Đặt

$$b_p = \sum_{t=p}^{p+k-1} a_t, \quad w(l, r) = \max_{l \leq p \leq r-k+1} b_p,$$

và coi $w(l, r) = -\infty$ nếu đoạn quá ngắn. Khi đó có thể kiểm tra

$$w(l, r - 1) + w(l + 1, r) - w(l + 1, r - 1) - w(l, r) \geq 0,$$

nên w thỏa bất đẳng thức tứ giác. Từ đó suy ra tính lồi trên của đáp án.

Về thuật toán, có thể dùng chia để trị với chập max-plus của các dãy lồi, hoặc dùng DP

$$f(p, i) = \max\{f(p - 1, i), f(p - k, i - 1) + b_{p-k+1}\}$$

kết hợp ngưỡng quyết định i_p và cây cân bằng bền vững để đạt $O(n \log^2 n)$ thời gian, $O(n)$ bộ nhớ.

4. Chứng minh bằng đại số

Khi không nhận ra mô hình thuật toán có sẵn, ta có thể quay lại định nghĩa lồi và phân tích công thức của hàm. Cách này thường dài hơn nhưng cho hiểu biết sâu hơn về cấu trúc bài toán.

4.1. Một ví dụ

Ví dụ 4.1 (High Performance Computer). Có n_A tác vụ loại A và n_B tác vụ loại B, cần phân cho p nút tính toán. Nút thứ i có tham số $t_i^A, t_i^B, k_i^A, k_i^B$: thực hiện liên tiếp x tác vụ loại A mất $t_i^A + k_i^A x^2$ nano giây, còn x tác vụ loại B mất $t_i^B + k_i^B x^2$ nano giây. Các nút chạy song song; cần tối thiểu hóa thời gian hoàn thành.

Thuật toán trực tiếp gồm hai tầng DP. Trước hết tính $f(i, a, b, c)$ là thời gian ngắn nhất để nút i xử lý a tác vụ A và b tác vụ B, với loại tác vụ cuối cùng là c . Chuyển trạng thái bằng cách liệt kê độ dài đoạn cuối:

$$\begin{aligned} f(i, a, b, 0) &= \min_{1 \leq x \leq a} \{f(i, a - x, b, 1) + t_i^A + k_i^A x^2\}, \\ f(i, a, b, 1) &= \min_{1 \leq x \leq b} \{f(i, a, b - x, 0) + t_i^B + k_i^B x^2\}. \end{aligned}$$

Sau đó gộp các nút:

$$g(i, a, b) = \min_{0 \leq x \leq a, 0 \leq y \leq b} \max\{g(i - 1, a - x, b - y), f(i, x, y)\}.$$

Phần gộp này quá đắt trên máy chấm thời điểm bài gốc, nên cần khai thác tính chất của f .

Nhận xét quan trọng là khi cố định a , hàm $f(i, a, b)$ theo b là đơn đỉnh. Khi nhị phân đáp án thời gian, tập các b thỏa $f(i, a, b) \leq ans$ là một đoạn liên tiếp. Do đó có thể tính với mỗi nút và mỗi a khoảng

$$h_{\min}(i, a) = \min\{b \mid f(i, a, b) \leq ans\}, \quad h_{\max}(i, a) = \max\{b \mid f(i, a, b) \leq ans\},$$

rồi gộp các khoảng bằng DP theo số tác vụ A. Phần này nhanh hơn, nhưng cần chứng minh tính đơn đỉnh.

Xét riêng một nút, viết gọn $f(a, b)$. Một phương án tối ưu chia các tác vụ A thành các đoạn liên tiếp độ dài $a_1, \dots, a_r$, các tác vụ B thành $b_1, \dots, b_s$, với $|r - s| \leq 1$. Nếu tồn tại hai đoạn A có độ dài chênh nhau ít nhất 2, chuyển một tác vụ từ đoạn dài sang đoạn ngắn làm thay đổi chi phí

$$k_i^A((a_{j_1} - 1)^2 + (a_{j_2} + 1)^2 - a_{j_1}^2 - a_{j_2}^2) = k_i^A(2 - 2(a_{j_1} - a_{j_2})) < 0,$$

mâu thuẫn với tối ưu. Vì vậy trong tối ưu, các đoạn cùng loại có độ dài chỉ chênh nhau nhiều nhất 1.

Nếu chia a tác vụ A thành r đoạn, viết $a = \alpha r + \beta$ với $0 \leq \beta < r$, chi phí nhỏ nhất của phần A là

$$f_A(a, r) = r t_i^A + k_i^A(r - \beta)\alpha^2 + k_i^A\beta(\alpha + 1)^2.$$

Hàm tương tự $f_B(b, s)$ cho tác vụ B. Bài toán một nút trở thành

$$f(a, b) = \min_{|r-s| \leq 1} \{f_A(a, r) + f_B(b, s)\}.$$

Các tính chất cần chứng minh là:

- (1) $f_A(a, r)$ theo a là lồi dưới và tăng khi cố định r ,
- (2) $f_A(a, r)$ theo r là lồi dưới khi cố định a ,
- (3) điểm cực tiểu theo r nằm gần cực tiểu của dạng liên tục.

Dạng liên tục thu được bằng cách cho độ dài đoạn không cần nguyên:

$$f_A^*(a, r) = r t_i^A + k_i^A r \left(\frac{a}{r}\right)^2 = r t_i^A + \frac{k_i^A a^2}{r}.$$

Dạng này gợi ý rõ ràng về lồi dưới và vị trí cực tiểu; phần còn lại là đưa nhận xét đó về trường hợp nguyên.

Khi cố định r , nếu tăng a thêm 1, một đoạn độ dài α trở thành $\alpha + 1$. Độ tăng là

$$\Delta(a) = k_i^A((\alpha + 1)^2 - \alpha^2) = k_i^A(2\alpha + 1),$$

không giảm theo a , nên $f_A(a, r)$ lồi dưới theo a .

Khi cố định a , xét các khối chia theo giá trị $\alpha = \lfloor a/r \rfloor$. Trong mỗi khối, $f_A(a, r)$ là hàm bậc nhất theo r và độ dốc bằng

$$t_i^A - k_i^A \alpha(\alpha + 1).$$

Khi r tăng thì α giảm, nên độ dốc không giảm. Cần kiểm tra thêm tại ranh giới các khối; có thể viết mỗi khối thành một đường thẳng $L_\alpha(r)$ và chứng minh

$$f_A(a, r) = \max_{1 \leq \alpha \leq a} L_\alpha(r).$$

Vì bao trên của các đường thẳng là lồi dưới, suy ra $f_A(a, r)$ lồi dưới theo r .

Từ đây, xét cặp số đoạn tối ưu (r, s) . Nếu $r < s$ thì cực tiểu riêng của A nằm bên trái r và cực tiểu riêng của B nằm bên phải s ; nếu $r > s$ thì ngược lại; nếu $r = s$ thì hai cực tiểu riêng cùng rơi vào vị trí đó. Khi cố định b và tăng a , cực tiểu phía A chỉ dịch sang phải, nên cặp tối ưu đi qua ba vùng theo thứ tự. Có thể chứng minh: nếu $f(a, b) \geq f(a + 1, b)$ thì trong phương án tối ưu của $f(a, b)$ phải có $r = a$. Trong vùng này

$$f(a, b) = f_A(a, a) + f_B(b, a + 1),$$

là tổng của một hàm tuyến tính và một hàm lồi dưới. Vì thế $f(a, b)$ theo a là ghép của một đoạn lồi dưới với một đoạn tăng, tức là đơn đỉnh. Tính đơn đỉnh dùng trong thuật toán nhị phân thời gian đã được chứng minh.

4.2. Một ví dụ khác

Ví dụ 4.2 (Dedenne). Một từ là xâu nhị phân không chứa hai số 0 liên tiếp. Từ điển là một tập từ không từ nào là tiền tố của từ khác. Nếu một xâu nhị phân là tiền tố của k từ trong từ điển thì chi phí của nó là

$$w(k) = \sum_{j=1}^k (1 + \lfloor \log_2 j \rfloor).$$

Chi phí của từ điển là tổng chi phí của mọi xâu nhị phân, kể cả xâu rỗng. Cần tối thiểu hóa chi phí của từ điển có n từ, với n rất lớn.

Mỗi xâu đóng góp chi phí tương ứng với một nút trong trie của từ điển. Gọi $f(n)$ là đáp án nhỏ nhất. Nếu cây con trái chứa k lá và cây con phải chứa $n - k$ lá, ta có

$$f(1) = w(1) = 1,$$

$$f(n) = w(n) + \min_{1 \leq k \leq n-1} \{w(k)[k > 1] + f(k) + f(n - k)\}.$$

DP trực tiếp mất $O(n^2)$.

Trước hết,

$$w(k) - w(k - 1) = 1 + \lfloor \log_2 k \rfloor,$$

nên w là lồi dưới. Bằng quy nạp đồng thời, có thể chứng minh $f(n)$ cũng tăng và lồi dưới, đồng thời điểm quyết định tối ưu k_n thỏa

$$0 \leq k_n - k_{n-1} \leq 1.$$

Ý tưởng là giả sử các kết luận đúng tới n_0 . Khi xét $n_0 + 1$, nếu chọn $k < k_{n_0}$ thì so với chọn k_{n_0} , phần chênh lệch ở trạng thái cũ không âm, còn chênh lệch sai phân của f cũng không âm. Vì vậy k không thể tốt hơn. Tương tự, mọi $k > k_{n_0} + 1$ cũng không tối ưu. Sau khi có quyết định đơn điệu, so sánh công thức sai phân của $f(n_0 + 1)$ và $f(n_0)$ sẽ cho $df(n_0 + 1) \geq df(n_0)$.

Nhờ đó DP có thể tối ưu xuống $O(n)$. Nhưng với n tới 10^{15} , vẫn không thể tính mọi trạng thái. Cần dùng thêm phân tích tiệm cận. Từ định nghĩa,

$$w(n) = \Theta(n \log n).$$

Nếu trong chuyển trạng thái luôn chọn gần $n/2$, ta được một chặn trên $O(n \log^2 n)$ cho $f(n)$. Nếu bỏ hạng $w(k)$ trong chuyển trạng thái để tạo một bài toán dễ hơn, lời giải tối ưu vẫn tách gần $n/2$, cho chặn dưới $\Omega(n \log^2 n)$. Do đó

$$f(n) = \Theta(n \log^2 n).$$

Vì f lồi dưới, sai phân của nó thỏa

$$df(n) = \Theta(\log^2 n).$$

Sai phân $df(n)$ vừa tăng vừa chỉ có $O(\log^2 n)$ giá trị khác nhau. Thuật toán cuối cùng duyệt các giá trị sai phân δ , tìm vị trí lớn nhất có $df(n) = \delta$ bằng tăng gấp bội và kiểm tra bằng thủ tục gần tam phân trên hàm lồi. Khi chọn hệ số gấp bội phù hợp, việc kiểm tra chỉ cần tra các đoạn sai phân đã biết. Độ phức tạp tổng thể là $O(\log^5 n)$. Trong thực tế, với giới hạn đề bài, có thể tiền xử lý các đầu đoạn rồi đưa vào chương trình nộp.

5. Chứng minh tính lồi trong hình học tính toán

Trong hình học tính toán, tính lồi thường không đến từ công thức DP mà đến từ bản thân đối tượng hình học: đa giác lồi, bao lồi, đường tròn, hoặc giao của các hình lồi. Khi bài toán xoay quanh những đối tượng đó, nên thử chứng minh bằng lập luận hình học trước khi đi vào công thức từng đoạn.

5.1. Một ví dụ

Ví dụ 5.1 (Easy Geometry). Cho một đa giác lồi n đỉnh trên mặt phẳng. Cần dựng trong đa giác một hình chữ nhật diện tích lớn nhất, các cạnh song song với trục tọa độ.

Thuật toán tự nhiên là tam phân lồng nhau: tam phân theo chiều rộng w của hình chữ nhật, rồi với w cố định tam phân theo tọa độ trái l . Sau khi biết l và $r = l + w$, có thể tìm biên trên và biên dưới bằng tìm kiếm trên hai đường bao của đa giác. Tính đúng đắn của hai lần tam phân phụ thuộc vào việc các hàm tương ứng là đơn đỉnh; thực ra chúng là lồi trên.

Với w cố định, gọi $h_w(l)$ là chiều cao lớn nhất của hình chữ nhật có biên trái l , và $s_w(l) = wh_w(l)$ là diện tích tương ứng. Hình học của $h_w(l)$ rất rõ: lấy giao của đa giác ban đầu với chính nó sau khi tịnh tiến sang trái w ; độ dài đoạn giao của đường thẳng $x = l$ với vùng giao này chính là $h_w(l)$. Giao của hai tập lồi vẫn lồi, nên độ dài lát cắt theo phương thẳng đứng là một hàm lồi trên theo l . Vì $s_w(l)$ chỉ nhân thêm hằng số w , nó cũng lồi trên. Do đó tam phân bên trong là hợp lệ.

Gọi

$$H(w) = \max_l h_w(l), \quad S(w) = wH(w).$$

Khi w tăng, vùng giao nói trên co lại, nên $H(w)$ không tăng.

Để chứng minh H và S lồi trên theo w , đặt $u(x)$ là biên trên của đa giác và $d(x)$ là biên dưới. Với đa giác lồi, u là lồi trên còn d là lồi dưới. Ta có

$$h_w(l) = \min\{u(l), u(l+w)\} - \max\{d(l), d(l+w)\},$$

hay tương đương là min của bốn hàm:

$$\begin{aligned} &u(l) - d(l), \\ &u(l) - d(l+w), \\ &u(l+w) - d(l), \\ &u(l+w) - d(l+w). \end{aligned}$$

Mỗi hàm trong bốn hàm này là lồi trên theo l .

Lấy $0 < a < b$, đặt $c = (a+b)/2$. Giả sử $H(a)$ đạt tại l_a và $H(b)$ đạt tại l_b , đặt $l_c = (l_a + l_b)/2$. Dùng tính lồi trên của u và lồi dưới của d cho từng trường hợp trong bốn biểu thức trên, ta có

$$2h_c(l_c) \geq h_a(l_a) + h_b(l_b).$$

Vì $H(c) \geq h_c(l_c)$, suy ra

$$2H(c) \geq H(a) + H(b),$$

tức H lồi trên.

Do $H(w)$ không tăng,

$$\begin{aligned} 2S(c) &= c \cdot 2H(c) \\ &\geq c(H(a) + H(b)) \\ &= \frac{a+b}{2}(H(a) + H(b)) \\ &= aH(a) + bH(b) + \frac{b-a}{2}(H(a) - H(b)) \\ &\geq aH(a) + bH(b) = S(a) + S(b). \end{aligned}$$

Vậy $S(w)$ cũng lồi trên, và tam phân ngoài là hợp lệ.

6. Tổng kết

Tính lỗi trong đề OI về bản chất vẫn là tính lỗi trong toán học. Khác biệt là trong bài toán toán học, chứng minh xong thường là kết thúc; còn trong OI, chứng minh chỉ là cơ sở để thiết kế và tối ưu thuật toán.

Bài viết trình bày ba hướng chứng minh: quy về mô hình quen thuộc, phân tích đại số trực tiếp, và khai thác hình học của đối tượng lỗi. Ba hướng này không bao quát mọi khả năng, nhưng là những cách xuất hiện nhiều trong quá trình giải bài. Khi gặp một tính chất lỗi mới, nên vừa quan sát dữ liệu để đoán dạng đúng, vừa tìm một chứng minh thật sự bằng một trong các con đường trên.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi, cảm ơn các huấn luyện viên đội tuyển quốc gia đã chỉ đạo. Xin cảm ơn gia đình đã đồng hành và ủng hộ tác giả. Xin cảm ơn thầy Dong Youming, thầy Li Xiang, Du Yuhao và Jiang Lingyu đã giúp đỡ trong quá trình viết bài. Xin cảm ơn Trường Trung học Nam Khai, Trùng Khánh và Trường THCS Nam Khai Rongqiao, Trùng Khánh đã bồi dưỡng tác giả, cùng các thầy cô và bạn học khác đã giúp đỡ.

Bài 3. Một lời giải dựa trên phân rã chuỗi dài cho bài toán lân cận trên cây

Liu Haifeng, Trường Trung học Tưởng niệm Trung Sơn

Tóm tắt

Bài viết xuất phát từ phân rã chuỗi dài, trình bày cách xử lý bốn dạng bài toán lân cận thường gặp trên cây tĩnh. Các cách xử lý bao gồm cả trường hợp offline và trường hợp bắt buộc online. Nhờ phân rã chuỗi dài, một phần bài toán lân cận được quy về truy vấn đoạn trên dãy và truy vấn đường trên cây trong thời gian tuyến tính.

1. Lời nói đầu

Với bài toán lân cận trên cây tĩnh, các công cụ thường gặp là phân trị điểm và static top tree. Phân rã chuỗi dài cũng là một kỹ thuật quen thuộc, thường dùng để tối ưu các bài toán liên quan đến độ sâu. Vì cấu trúc của lân cận cũng gắn chặt với độ sâu, ta có thể thử dùng phân rã chuỗi dài để xử lý các loại lân cận khác nhau trên cây.

2. Kiến thức chuẩn bị

Bài viết chỉ cần các kiến thức cơ bản về cây.

3. Quy ước và định nghĩa

Nếu không nói rõ, cây trong bài là cây có gốc gồm n đỉnh, gốc là đỉnh 1, mọi cạnh có độ dài 1, và không có thao tác sửa đổi.

Để tiện trình bày, “nửa nhóm” dưới đây mặc định là nửa nhóm giao hoán, còn “nhóm” mặc định là nhóm giao hoán, trừ khi có ghi chú riêng.

Định nghĩa 3.1 (độ sâu). Độ sâu của một đỉnh là khoảng cách ngắn nhất từ đỉnh đó đến gốc cộng 1. Ký hiệu f_{a_x} là cha của đỉnh x .

3.1. Định nghĩa cơ bản của phân rã chuỗi dài

Định nghĩa 3.2 (chiều cao). Chiều cao của một đỉnh là khoảng cách từ đỉnh đó đến đỉnh xa nhất trong cây con của nó, cộng 1. Lá có chiều cao 1; chiều cao của x được ký hiệu là h_x .

Định nghĩa 3.3 (con dài). Con dài của một đỉnh là con có chiều cao lớn nhất. Nếu có nhiều con cùng đạt cực đại thì chọn tùy ý; lá không có con dài.

Định nghĩa 3.4 (con ngắn). Các con không phải con dài được gọi là con ngắn.

Định nghĩa 3.5 (chuỗi dài). Giữ lại, với mỗi đỉnh không phải lá, chỉ cạnh nối tới con dài của nó. Các thành phần đường thu được gọi là chuỗi dài; một chuỗi có thể chỉ gồm một đỉnh. Độ dài của chuỗi là số đỉnh trong chuỗi.

Định lý 3.1 (định lý cơ bản của phân rã chuỗi dài). Gọi $f(x)$ là tổng chiều cao của các con ngắn của x . Khi đó

$$\sum_{i=1}^n f(i) = n - \text{chiều cao của gốc}.$$

Chứng minh. Tách đóng góp của tổng trên: mọi chuỗi dài trừ chuỗi chứa gốc được tính đúng một lần, còn chuỗi chứa gốc không được tính. Vì vậy tổng đúng bằng số đỉnh không nằm trên chuỗi dài của gốc, tức $n - \text{chiều cao của gốc}$.

3.2. Các định nghĩa liên quan tới lân cận

Tên của một số khái niệm trong phần này do tác giả đặt để tiện trình bày.

Định nghĩa 3.6 (lân cận). Lân cận biểu diễn bởi cặp (x, d) là tập các đỉnh có khoảng cách tới x không quá d .

Định nghĩa 3.7 (lân cận trong). Lân cận trong biểu diễn bởi (x, d) là tập các đỉnh nằm trong cây con của x và có khoảng cách tới x không quá d . Một số tài liệu cũng gọi đây là lân cận có hướng.

Định nghĩa 3.8 (lân cận ngoài). Lân cận ngoài biểu diễn bởi (x, d) là tập các đỉnh không nằm trong cây con của x và có khoảng cách tới x không quá d .

Định nghĩa 3.9 (lân cận chuỗi). Với bộ ba (x, y, d) , trong đó y là tổ tiên của x , lân cận chuỗi là tập các đỉnh thỏa:

1. nằm ngoài cây con của x nhưng nằm trong cây con của y ;
2. tồn tại một đỉnh trên đường từ x đến y có khoảng cách tới đỉnh đang xét không quá d .

Hai phần tiếp theo xử lý truy vấn thông tin nhóm và nửa nhóm trên các loại lân cận trong trường hợp offline.

4. Truy vấn thông tin nhóm

Trong phần này, để đơn giản, ta xét bài toán tính tổng trọng số đỉnh trên các loại lân cận. Vì thông tin thuộc nhóm, ta được phép dùng phép trừ hoặc phần tử nghịch đảo. Mục tiêu là xử lý offline mọi truy vấn trong thời gian tuyến tính.

4.1. Lân cận trong

Với một chuỗi dài có độ dài m và đỉnh đầu chuỗi là x , trước hết tiền xử lý tổng trọng số các đỉnh trong cây con của x có khoảng cách tới x bằng i . Theo định lý cơ bản của phân rã chuỗi dài, phần này có thể làm trực tiếp với tổng thời gian tuyến tính.

Sau đó xét từng chuỗi dài. Với mỗi đỉnh x , tiền xử lý một mảng có độ dài bằng chiều cao lớn nhất trong các con ngắn của x ; phần tử thứ i là tổng trọng số các đỉnh nằm trong cây con của các con ngắn và cách x đúng i . Như vậy toàn bộ cây có thể xem như nhiều chuỗi dài, mỗi đỉnh trên chuỗi còn có thể treo thêm một chuỗi phụ chứa thông tin của các con ngắn; tổng kích thước các phần phụ vẫn là $O(n)$.

Với mỗi chuỗi, quét từ đáy lên đỉnh. Khi đang ở vị trí i , các thay đổi chỉ ảnh hưởng tới vị trí $i - 1$ và một đoạn phía sau; tổng số thay đổi trên mọi chuỗi là $O(n)$. Treo truy vấn vào các điểm tương ứng trên chuỗi thì bài toán trở thành nhiều truy vấn tổng đoạn. Do có thể duy trì tổng hậu tố, ta đạt thời gian tuyến tính.

Về bản chất, cấu trúc cần truy vấn giống một lưới: cho điểm (x, y) , cần tổng các điểm nằm ở vùng bên trái phía dưới nó. Một cách nhìn khác là tìm nhanh vị trí có giá trị đầu tiên (x', y) nằm dưới (x, y) , rồi dùng đáp án đã tiền xử lý tại vị trí đó cộng với phần các hàng $[x, x')$. Với truy vấn online bắt buộc, việc tìm vị trí này sẽ được chuyển thành một bài toán riêng ở phần 7.

4.2. Lân cận

Lân cận ngoài có thể tính bằng lân cận trừ lân cận trong, nên trước hết chỉ cần xét lân cận thông thường. Với lân cận (x, d) , nếu thay nó bằng $(fa_x, d - 1)$ thì trạng thái của các đỉnh ngoài cây con x không đổi. Nếu $d \geq h_x + 1$, cả hai lân cận đều chứa toàn bộ cây con x , nên phép thay không làm đổi đáp án. Ta có thể liên tục nhảy lên cha cho tới khi không thể nhảy nữa.

Định nghĩa 4.1 (thao tác nhảy chuỗi). Với (x, d) , tìm tổ tiên gần nhất y của x sao cho độ dài chuỗi dài chứa y nhân 2 không nhỏ hơn d . Nhảy tới y và giảm d theo chênh lệch độ sâu. Cặp mới (y, d') thỏa

$$d' \leq 2 \cdot \text{độ dài chuỗi dài chứa } y.$$

Nếu thay thao tác nhảy chuỗi bằng việc nhảy từng bước từ (x, d) sang $(fa_x, d - 1)$, thì trước khi tới y luôn có $d \geq h_x + 1$. Lý do là với mỗi đỉnh z trên đường đi từ x tới y , trừ y , độ dài chuỗi dài chứa z vẫn đủ lớn so với phần d đã giảm, nên điều kiện bao trọn cây con vẫn đúng.

Bài toán phụ còn lại là: cho x, k , tìm tổ tiên gần nhất y sao cho chuỗi dài chứa y có độ dài ít nhất k . Trong offline, có thể DFS và mở n ngăn xếp; ngăn xếp thứ i lưu chuỗi cần nhảy tới khi $d = i$. Khi DFS vào một con ngắn có chuỗi dài số k , độ dài m , đẩy k vào các ngăn xếp $1, \dots, m$, và khi rời khỏi cây con thì pop lại. Nhờ đó ta biết truy vấn sau khi nhảy sẽ rơi vào chuỗi nào; duy trì thêm vài thông tin đơn giản trong DFS sẽ xác định được cặp (y, d') .

Với đỉnh đầu x của chuỗi dài độ dài m , tiền xử lý các lân cận ngoài $(x, 1), (x, 2), \dots, (x, 2m)$. Nếu một truy vấn ngoài tại chuỗi này có $d \leq 2m$, ta tách đóng góp thành phần ngoài cây con của đỉnh đầu chuỗi và phần trong cây con đỉnh đầu chuỗi; phần thứ nhất đã tiền xử lý, phần thứ hai xử lý như lân cận trong.

Để tiện xử lý lân cận ngoài của đỉnh đầu chuỗi x , gọi $y = fa_x$. Lân cận ngoài (x, d) có thể viết thành lân cận ngoài $(y, d - 1)$ cộng với lân cận chuỗi $(x, y, d - 1)$. Vì đang ở trường hợp nhóm, các phần chồng nhau có thể xử lý bằng bao hàm–loại trừ. Toàn bộ tiền xử lý là $O(n)$, và Q truy vấn offline được xử lý trong $O(n + Q)$.

4.3. Lân cận chuỗi

Cách làm tương tự lân cận thường. Đặt $A(x, d)$ là tổng của lân cận chuỗi $(x, 1, d)$ cộng với lân cận trong (x, d) . Nếu d lớn hơn độ dài chuỗi dài chứa x , thì

$$A(x, d) = A(fa_x, d).$$

Vì vậy ta lại dùng một thao tác nhảy chuỗi, nhưng lần này cần tổ tiên gần nhất y sao cho d không vượt quá độ dài chuỗi dài chứa y .

Với mỗi chuỗi dài có đỉnh đầu x và độ dài m , tiền xử lý

$$(x, 1, 1), (x, 1, 2), \dots, (x, 1, m).$$

Ý tưởng là chuyển $(x, 1, i)$ thành lân cận chuỗi $(fa_x, 1, i)$, cộng lân cận trong (fa_x, i) , rồi trừ lân cận trong $(x, i - 1)$.

Nếu một truy vấn $(x, 1, d)$ thỏa d không vượt quá độ dài chuỗi chứa x , gọi y là đỉnh đầu chuỗi của x . Ta tách truy vấn thành (x, y, d) và $(y, 1, d)$. Phần sau đã được tiền xử lý, còn phần trước là truy vấn đóng góp trên một đoạn của chuỗi, có thể xử lý như cấu trúc lưới ở phần lân cận trong. Cuối cùng lấy $A(x, d)$ trừ lân cận trong (x, d) để thu được lân cận chuỗi cần tìm.

5. Truy vấn thông tin nửa nhóm

Phần này nhằm quy các truy vấn nửa nhóm giao hoán về những bài toán chuẩn hơn trong $O(n + Q)$: lân cận trong và ngoài được quy về truy vấn nửa nhóm trên đoạn của dãy, còn lân cận chuỗi được quy về truy vấn nửa nhóm trên đường của cây. Vì không có nghịch đảo, không thể dùng phép trừ như phần trước.

5.1. Lân cận trong

Quay lại mô hình lưới của lân cận trong. Với truy vấn (x, y) , tìm vị trí đầu tiên (x', y) nằm bên dưới cùng cột và có giá trị, rồi dùng đáp án tiền xử lý tại đó kết hợp với thông tin của các hàng $[x, x')$. Việc tiền xử lý đáp án tại các vị trí có giá trị không cần dùng phép trừ. Trong offline, tìm x' bằng cách quét từ dưới lên và dùng thùng cho từng cột y để ghi vị trí nhỏ nhất đã gặp.

5.2. Lân cận

Ta vẫn thực hiện thao tác nhảy chuỗi như trường hợp nhóm. Sau khi nhảy xong, truy vấn tương đương với ghép một lân cận ngoài và một lân cận trong. Điểm khác biệt là mọi phần phải được tách thành các đoạn không giao nhau hoặc các đường cây độc lập, thay vì trừ phần thừa.

5.3. Lân cận ngoài

Trong tiền xử lý của trường hợp nhóm, phần dùng phép trừ là truy vấn thông tin của fa_x sau khi bỏ cây con x . Không cần nghịch đảo, ta có thể xử lý bằng cách tách các con. Gọi y là con dài của fa_x . Tiền xử

lý lân cận trong của y với các bán kính cần thiết, rồi chỉ còn phải ghép đóng góp của các con ngắn khác của fa_x . Các truy vấn đối với các con ngắn có tổng quy mô tuyến tính nên có thể làm trực tiếp.

Khi truy vấn ngoài (x, d) quá lớn so với chuỗi hiện tại, ta nhảy tới một chuỗi i . Tìm tổ tiên gần nhất y sao cho độ dài chuỗi chứa y nhân 2 đủ lớn, đặt $d' = d - \text{dist}(x, y)$. Ta cần ghép:

- lân cận ngoài (y, d') , đã được tiền xử lý;
- phần trong cây con của y nhưng không thuộc nhánh đi xuống x .

Nếu gọi z là con của y nằm trên đường tới x , phần thứ hai chính là lân cận chuỗi (z, y, d') .

Có một cách cài đặt đơn giản hơn: với mỗi chuỗi dài, đánh số các con ngắn theo độ sâu, con nông hơn có số nhỏ hơn. Với truy vấn (x, d) , xác định y, z, d' và số thứ tự của nhánh z . Sau đó ghép đóng góp của các con ngắn có số lớn hơn z , các con ngắn có số nhỏ hơn z , và phần ngoài cây con của đỉnh đầu chuỗi. Mỗi nhóm lại quy về kiểu truy vấn ở phần 5.1. Nếu từ a nhảy tới b , còn cần cộng thông tin của cây con b sau khi bỏ cây con a ; trên thứ tự DFS, phần này là hợp của hai đoạn. Do đó lân cận trong và ngoài offline được quy về truy vấn đoạn trên dãy trong $O(n + Q)$.

5.4. Lân cận chuỗi

Nếu chuỗi kết thúc tại gốc, cách làm về bản chất giống lân cận ngoài. Trước hết xét trường hợp x, y nằm trên cùng một chuỗi dài. Truy vấn có dạng lấy một dải hàng $[l, r]$ và d cột đầu trong mô hình lưới. Vì không có phép trừ, ta tìm vị trí có giá trị đầu tiên ở cột d phía dưới hàng l , và vị trí có giá trị đầu tiên phía trên hàng r . Với mỗi cột, duy trì thông tin nửa nhóm của các khoảng giữa những vị trí có giá trị; tổng số vị trí có giá trị là $O(n)$.

Đặt g_x là chiều cao lớn nhất trong các con ngắn của x , và $g_x = 0$ nếu không có con ngắn. Xây một rừng sinh có n lớp: với mỗi đỉnh x của cây gốc, tạo các bản sao $(x, 1), \dots, (x, g_x)$. Tổng số bản sao không vượt quá n . Ở lớp i , nối (x, i) tới (y, i) , trong đó y là tổ tiên gần nhất của x thỏa $g_y \geq i$. Trọng số của cạnh này chính là thông tin nửa nhóm của lân cận chuỗi tương ứng.

Sau khi biết thông tin trên các cạnh của rừng sinh, một truy vấn lân cận chuỗi có thể được tách thành phần đã nhảy qua và phần còn lại trong cùng chuỗi. Nếu hai đầu không cùng chuỗi, tìm con ngắn đầu tiên k trên đường từ y xuống x , dùng truy vấn đường trên rừng sinh để lấy (x, fa_k, d) , rồi phần (fa_k, y, d) nằm trên cùng chuỗi và xử lý bằng cách ở trên.

Cách tính thông tin cho mọi cạnh của rừng sinh cũng tuyến tính. Nếu ở lớp i , x nối tới y , thì các đỉnh nằm giữa x và y có giá trị $g \leq i$. Bài toán quy về một cấu trúc dữ liệu hỗ trợ:

- thêm thông tin nửa nhóm cho một tiền tố, đồng thời thêm một thông tin cố định cho phần sau tiền tố;
- truy vấn thông tin của một tiền tố rồi xóa phần đã truy vấn.

Vì tổng lượng sửa đổi là $O(n)$, chỉ cần dùng lazy tag và đẩy tag dần về sau khi thực hiện thao tác tiền tố; tổng thời gian vẫn tuyến tính.

6. Các trường hợp khác

Một số bài toán có thông tin không thể tách thành từng lớp khoảng cách đúng bằng i ; ta chỉ duy trì được thông tin của tập các đỉnh trong cây con x có khoảng cách tới x không quá i . Nếu phép chuyển trạng thái có thể ghép nhanh và có tính kết hợp, phân rã chuỗi dài vẫn xử lý được.

Ví dụ 6.1. Cho cây có trọng số đỉnh. Mỗi truy vấn cho x, d , cần tìm trọng số lớn nhất của tập độc lập trong lân cận trong (x, d) .

Với mỗi chuỗi dài, tính thông tin cho lân cận có bán kính i quanh đỉnh đầu chuỗi. Với mỗi đỉnh x , trước hết tính phần cây con của x sau khi bỏ con dài, rồi dùng DP của con dài để chuyển. Khi quét một chuỗi, sửa đổi tại vị trí cách đỉnh đầu chuỗi i tương đương với cập nhật trực tiếp một đoạn tiền tố ngắn, rồi nhân cùng một ma trận cho phần còn lại. Giống phần 5.4, lazy tag cho phép đẩy các phép nhân này về sau trong tổng thời gian tuyến tính.

Khi truy vấn một vị trí, nếu phía trước còn lazy tag chưa đẩy, giá trị đọc được chưa phải đáp án cuối cùng; cần ghép thêm đóng góp của các hàng trước nó. Nhìn từ góc khác, mỗi lazy tag chính là thông tin của phần cây con sau khi bỏ con dài, nên bản chất vẫn giống cách chuyển DP trên chuỗi dài.

Các loại lân cận còn lại cũng xử lý tương tự, chỉ là chi tiết cài đặt rườm rà hơn.

7. Bất buộc online

Phần này giả sử đã có cấu trúc LCA online $O(n)$ -tiền xử lý, $O(1)$ -truy vấn và cấu trúc tổ tiên cấp k online với cùng độ phức tạp. Mục tiêu là chuyển các bài toán lân cận online về truy vấn nửa nhóm online trên đoạn hoặc trên đường cây.

7.1. Bài toán tiền nhiệm trên dãy

Xét bài toán phụ: cho dãy số nguyên dương $a_1, \dots, a_n$, mỗi truy vấn cho i, x , cần tìm chỉ số lớn nhất $j \leq i$ sao cho $a_j \geq x$, hoặc báo không tồn tại. Gọi $S = \sum_i a_i$; yêu cầu $O(n + S)$ tiền xử lý và $O(1)$ mỗi truy vấn online.

Nếu offline, chỉ cần quét chỉ số và dùng thùng lưu vị trí cuối cùng đã gặp cho mỗi độ cao. Với online, xây đồ thị có S đỉnh: với mỗi i , tạo $(i, 1), (i, 2), \dots, (i, a_i)$. Với mỗi đỉnh (x, y) , tìm $z \leq x$ lớn nhất sao cho $(z, y + 1)$ tồn tại, rồi nối (x, y) tới $(z, y + 1)$ nếu có. Đồ thị thu được là một rừng hướng vào trong. Truy vấn (i, x) trở thành tìm tổ tiên cấp $x - 1$ của $(i, 1)$, nên có thể trả lời trong $O(1)$.

7.2. Bài toán nhảy chuỗi

Trong lân cận ngoài và lân cận chuỗi, ta thường cần: với đỉnh x , tìm tổ tiên gần nhất y sao cho chuỗi dài chứa y có độ dài ít nhất k . Cách offline đã nêu ở phần 4.2; với online, co mỗi chuỗi dài thành một đỉnh. Chuỗi a nối lên chuỗi b nếu cha của đỉnh đầu chuỗi a nằm trên chuỗi b . Khi đó bài toán trở thành tiền nhiệm trên cây.

Cách dựng tương tự bài toán tiền nhiệm trên dãy: với chuỗi x có độ dài a_x , tạo $(x, 1), \dots, (x, a_x)$. Với (x, y) , tìm tổ tiên gần nhất z của chuỗi x sao cho $(z, y + 1)$ tồn tại, rồi nối (x, y) tới $(z, y + 1)$. Truy vấn tại chuỗi i chỉ cần tìm tổ tiên cấp $k - 1$ của $(i, 1)$. Kết quả cho biết truy vấn dừng ở chuỗi nào; để biết chính xác đỉnh dừng, lấy LCA của x với đáy của chuỗi đó.

7.3. Cách xử lý các lân cận

Ta vẫn muốn dùng $O(n)$ tiền xử lý và $O(1)$ để chuyển truy vấn về truy vấn nửa nhóm online trên đoạn hoặc trên đường cây. Với lân cận trong, nhiệm vụ là: cho (x, y) , tìm vị trí đầu tiên nằm dưới cùng cột và có giá trị. Bài toán này cũng chuyển thành tổ tiên cấp k trên cây. Với mỗi vị trí có giá trị (x, y) , tìm i nhỏ nhất sao cho $i \geq x$ và $(i, y + 1)$ có giá trị, rồi nối (x, y) tới $(i, y + 1)$. Khi truy vấn, bắt đầu từ (x, x) và lấy tổ tiên cấp $y - x$.

Các phần còn lại của lân cận trong đều được giải bằng truy vấn nửa nhóm trên dãy hoặc trên cây. Với lân cận ngoài và lân cận chuỗi, trước hết xử lý thao tác nhảy chuỗi online như phần 7.2; sau đó các mảnh còn lại được tách và truy vấn bằng cùng các cấu trúc nửa nhóm.

8. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn Trường Trung học Tưởng niệm Trung Sơn và Trường Sanxin đã cung cấp môi trường học tập tin học tốt. Xin cảm ơn gia đình, bạn bè và huấn luyện viên đã ủng hộ, động viên tác giả. Xin cảm ơn Zhao Haikun, Xu Qiwen và Pang Jiayi đã góp ý quý báu cho bài viết.

Tài liệu tham khảo

1. Liu Rujia. *Nghệ thuật thuật toán và thi đấu tin học*.

Bài 4. Ứng dụng nguyên lý chuyển vị trong một lớp bài toán quy hoạch động

Xu Xiaoyang, Trường Trung học số 1 trực thuộc Đại học Sư phạm Hoa Trung

Tóm tắt

Nguyên lý chuyển vị (*Transposition Principle*) là tư tưởng dùng ma trận chuyển vị để viết lại thuật toán của một biến đổi tuyến tính. Nó có thể tối ưu hiệu quả độ phức tạp thời gian và bộ nhớ của một số bài toán biến đổi tuyến tính. Bài viết đưa nguyên lý này vào quy hoạch động, kết hợp với một số thao tác trên cây đoạn, rồi thông qua hai ví dụ trình bày lợi thế của cách nhìn chuyển vị khi giải các bài toán DP cần tối ưu bằng cây đoạn.

1. Mở đầu

Từ thế kỷ trước, nguyên lý chuyển vị đã được dùng trong tối ưu thuật toán biến đổi tuyến tính. Nó liên hệ chặt chẽ một thuật toán biến đổi tuyến tính với thuật toán của biến đổi chuyển vị. Trước đó, người ta thường tối ưu một biến đổi và chuyển vị của nó một cách riêng rẽ, ví dụ nhân đa thức và bài toán chuyển vị tương ứng. Nguyên lý chuyển vị cho biết hai bài toán có thể được giải trong gần như cùng độ phức tạp thời gian và bộ nhớ. Vì vậy, khi tối ưu được một thuật toán biến đổi tuyến tính, ta cũng có thể thu được thuật toán cho chuyển vị của nó.

Với bài toán đơn giản, thường có thể tối ưu trực tiếp. Giá trị của nguyên lý chuyển vị thể hiện rõ hơn ở những bài toán khó, nhất là khi thuật toán có bản chất tuyến tính nhưng dạng trực tiếp không dễ nhìn ra. Hiện tại, các ứng dụng trong OI chủ yếu liên quan đến hàm sinh và phép toán đa thức. Bài viết này bàn về một hướng ít được nhắc tới hơn: dùng chuyển vị trong DP tối ưu bằng hợp nhất hoặc tách cây đoạn.

2. Nguyên lý chuyển vị

2.1. Giới thiệu

Định nghĩa 2.1 (thuật toán biến đổi tuyến tính). Với ma trận A kích thước $m \times n$, một thuật toán có thể tính

$$b = Aa$$

với mọi vector cột $a \in R^n$ được gọi là thuật toán biến đổi tuyến tính ứng với A .

Nhân ma trận trực tiếp là một ví dụ hiển nhiên, nhưng thuật toán biến đổi tuyến tính không nhất thiết phải cài bằng nhân ma trận thô. Ví dụ, thuật toán tính tổng hậu tố có thể xem là nhân với ma trận tam giác trên toàn số 1. Cách cài đặt của thuật toán chỉ phụ thuộc vào ma trận A , không phụ thuộc giá trị cụ thể của vector đầu vào.

Định nghĩa 2.2 (chuyển vị của thuật toán). Với thuật toán f ứng với ma trận A , thuật toán g ứng với ma trận A^T được gọi là chuyển vị của f . Nghĩa là với mọi vector cột $a' \in R^m$, thuật toán g tính

$$b' = A^T a'.$$

Vector a' ở đây là một đầu vào độc lập, không có quan hệ số trị với vector a của thuật toán gốc.

Vẫn với ví dụ tổng hậu tố, ma trận chuyển vị là ma trận tam giác dưới toàn số 1, tức thuật toán tổng tiền tố. Do đó có thể xem tổng tiền tố là chuyển vị của tổng hậu tố.

Theo nguyên lý chuyển vị, có thể viết lại một thuật toán biến đổi tuyến tính thành thuật toán cho biến đổi chuyển vị mà không đổi bậc độ phức tạp. Nếu bài toán chuyển vị dễ tối ưu hơn, ta có thể tối ưu nó rồi chuyển ngược để giải bài toán ban đầu.

2.2. Viết lại thuật toán

Để tiện xử lý, có thể thêm hàng và cột để biến A thành một ma trận vuông khả nghịch:

$$\begin{pmatrix} A & I_m \\ I_n & 0 \end{pmatrix}.$$

Đồng thời thêm m phần tử vào vector đầu vào và đặt chúng bằng 0; phần mở rộng không ảnh hưởng đến m phần tử đầu của kết quả, còn n phần tử sau chỉ khôi phục lại đầu vào ban đầu. Vì vậy phần mở rộng không làm đổi bản chất thuật toán hay bậc độ phức tạp. Nếu không nói rõ, ta có thể giả sử A là ma trận vuông khả nghịch.

Mọi ma trận khả nghịch có thể phân tích thành tích các ma trận sơ cấp. Ba thao tác sơ cấp tương ứng là:

1. hoán đổi a_i và a_j , ký hiệu $a_i \leftrightarrow a_j$;
2. nhân a_i với hằng số k , ký hiệu $a_i \leftarrow ka_i$;
3. cộng ka_j vào a_i , ký hiệu $a_i \leftarrow a_i + ka_j$.

Hai thao tác đầu khi chuyển vị vẫn giữ nguyên. Thao tác thứ ba chuyển thành

$$a_j \leftarrow a_j + ka_i.$$

Vì $A^T = V_k^T V_{k-1}^T \cdots V_1^T$, muốn lấy chuyển vị của thuật toán thì đổi từng thao tác thành thao tác chuyển vị tương ứng rồi thực hiện theo thứ tự ngược.

3. Nguyên lý chuyển vị và quy hoạch động

3.1. Tư tưởng cơ bản

Muốn dùng nguyên lý chuyển vị, trước hết cần mô tả bài toán dưới dạng một biến đổi tuyến tính. Sau đó dựa trên ý nghĩa cụ thể của ma trận A , ta diễn giải bài toán chuyển vị $b' = A^T a'$. Nếu bài toán chuyển vị có thể giải bằng DP hoặc có DP dễ tối ưu hơn, ta chuyển vị quá trình đó để thu được lời giải cho bài toán gốc.

3.2. Tối ưu quy hoạch động

Với nhiều bài toán, vẫn có thể thiết kế trạng thái và chuyển trực tiếp mà không cần nguyên lý chuyển vị. Nhưng trong các bài cần tối ưu DP bằng cấu trúc dữ liệu phức tạp, cách thiết kế trực tiếp có thể rườm rà và thiếu tự nhiên. Khi đó, nếu bài toán có thể viết thành biến đổi tuyến tính, nguyên lý chuyển vị giúp chuyển bài toán sang dạng dễ nhìn hơn.

Những bài toán chuyển vị hiện thường gặp nhất liên quan tới:

- hàm sinh và đa thức;
- cây đoạn.

Bài viết tập trung vào nhóm thứ hai: các DP cần tối ưu bằng thao tác trên cây đoạn.

4. Chuyển vị của một số thao tác trên cây đoạn

Trước khi chuyển vị một thuật toán DP tối ưu bằng cây đoạn, cần hiểu chuyển vị của các thao tác cây đoạn là gì. Khi làm điều này, phải mô tả thuật toán cây đoạn thành một biến đổi tuyến tính: dữ liệu đầu vào nào được đặt trong vector, dữ liệu nào được xem là phần cố định của ma trận A , và đầu ra nằm ở đâu.

4.1. Cộng đoạn, hỏi điểm

Xét dãy a_i độ dài n , ban đầu toàn 0, hỗ trợ:

1. $l\ r\ x$: cộng x cho mọi a_i với $i \in [l, r]$;
2. $q\ y$: ghi giá trị a_q vào y .

Các giá trị x là đầu vào, còn l, r, q được xem là cố định trong ma trận của biến đổi. Với mỗi truy vấn y_i , nó là tổng của những x_j có thao tác cộng xuất hiện trước truy vấn và thỏa $q_i \in [l_j, r_j]$. Do đó toàn bộ quá trình là một biến đổi tuyến tính.

Chuyển vị của thao tác thô. Nếu cài thô, thao tác cộng đoạn là nhiều phép

$$a_t \leftarrow a_t + x_i \quad (t = l_i, \dots, r_i),$$

còn thao tác hỏi điểm là

$$y_i \leftarrow y_i + a_{q_i}.$$

Sau khi chuyển vị và đảo thứ tự:

- thao tác $l\ r\ x$ trở thành lấy tổng $a_l + a_{l+1} + \dots + a_r$ và cộng vào x ;
- thao tác $q\ y$ trở thành cộng y vào a_q .

Nói cách khác, chuyển vị của “cộng đoạn, hỏi điểm” là “cộng điểm, hỏi tổng đoạn”.

Chuyển vị của thao tác trên cây đoạn. Dùng đánh dấu vĩnh viễn trên cây đoạn: mỗi nút pos lưu s_{pos} , nghĩa là mọi vị trí trong đoạn của nút đều được cộng thêm s_{pos} . Vector trung gian không còn là n giá trị a_i , mà là $2n - 1$ giá trị s_{pos} .

Thao tác cộng đoạn tách $[l_i, r_i]$ thành các nút cây đoạn S_i , rồi với mỗi $pos \in S_i$ thực hiện $s_{pos} \leftarrow s_{pos} + x_i$. Thao tác hồi điểm lấy các nút trên đường chứa q_i , gọi là S'_i , rồi thực hiện $y_i \leftarrow y_i + s_{pos}$ với mọi $pos \in S'_i$. Chuyển vị cho ta:

- thao tác l_i, r_i, x_i : cộng các s_{pos} của $pos \in S_i$ vào x_i ;
- thao tác q_i, y_i : cộng y_i vào mọi s_{pos} trên đường chứa q_i .

Ý nghĩa thực tế vẫn đúng là “cộng điểm, hồi tổng đoạn”. Như vậy có hai con đường để lấy chuyển vị của một thao tác cây đoạn: chuyển vị trực tiếp thao tác trên cây đoạn, hoặc trước hết chuyển vị thao tác thô rồi tối ưu bài toán mới bằng cây đoạn. Nội dung của hình gốc chỉ minh họa hai con đường này; bài viết không cần chèn lại hình để dùng kết luận.

4.2. Hai cách giải chuyển vị của thao tác cây đoạn

Từ phân tích trên, có hai phương pháp:

1. phân tích trực tiếp các thao tác trên cây đoạn rồi lấy chuyển vị;
2. tìm chuyển vị của thao tác thô, sau đó dùng cây đoạn tối ưu bài toán chuyển vị.

Phương pháp thứ nhất phức tạp hơn khi phân tích nhưng áp dụng được cho mọi thao tác trên cây đoạn. Phương pháp thứ hai dễ hơn khi thuật toán thô đơn giản, nhưng khi thuật toán gốc phức tạp, việc diễn giải và tối ưu bài toán chuyển vị có thể khó hơn, thậm chí cần thêm chứng minh về độ phức tạp và tính đúng.

4.3. Chuyển vị của hợp nhất cây đoạn

Phần này tạm không xét giá trị số, chỉ xét quá trình đảo ngược thao tác hợp nhất cây đoạn. Trong các bài toán thực tế, ta thường chạy hợp nhất cây đoạn bình thường rồi xử lý ngược lại; quá trình ngược này rất giống thao tác rollback và được gọi là tách cây đoạn. Khái niệm này khác với tách cây đoạn thông thường theo giá trị. Ở đây hai cây sau khi tách có thể có miền giá trị trùng nhau, vì chúng là hai cây trước khi hợp nhất.

Khi hợp nhất hai cây T_1, T_2 , xét hai gốc rt_1, rt_2 :

- nếu một trong hai rỗng, trả về cây không rỗng;
- nếu cả hai không rỗng, hợp nhất con trái với con trái, con phải với con phải, cập nhật thông tin và trả về rt_1 .

Muốn tách cây T trở lại T_1, T_2 , ta đảo quá trình trên: nút sinh ra từ trường hợp một bên rỗng được trả lại đúng cây con cũ; nút sinh ra từ trường hợp cả hai không rỗng thì đệ quy tách hai con rồi cập nhật hai nút gốc. Nếu cần rollback, chỉ cần ghi lại những thông tin bị sửa khi hợp nhất; thời gian và bộ nhớ vẫn giữ $O(n \log n)$.

4.4. Hợp nhất cây đoạn để tối ưu MAX-convolution

4.4.1. MAX-convolution. Với dãy viết thành chuỗi lũy thừa hình thức

$$F = \sum_{i=0}^n f_i x^i, \quad G = \sum_{i=0}^n g_i x^i,$$

định nghĩa $H = F \otimes G$ nếu với mọi k :

$$h_k = \sum_{\max(i,j)=k} f_i g_j.$$

4.4.2. Chuyển vị của MAX-convolution. Xem F là đầu vào, G là hằng số tạo ma trận A , và H là đầu ra. Chuyển vị được ký hiệu $F = H \otimes^T G$. Theo định nghĩa, thao tác gốc với mỗi cặp i, j là

$$h_{\max(i,j)} \leftarrow h_{\max(i,j)} + f_i g_j.$$

Chuyển vị của nó là

$$f_i \leftarrow f_i + h_{\max(i,j)} g_j.$$

Do đó

$$f_i = \sum_{j=0}^n h_{\max(i,j)} g_j = h_i \sum_{j=0}^i g_j + \sum_{j=i+1}^n h_j g_j.$$

4.4.3. Tính MAX-convolution. Để tính $H = F \otimes G$, duyệt $i = 0, \dots, n$, đồng thời duy trì

$$s_f = \sum_{j < i} f_j, \quad s_g = \sum_{j < i} g_j.$$

Nếu $f_i = g_i = 0$ thì $h_i = 0$. Nếu chỉ một trong hai khác 0, h_i bằng tích của giá trị khác 0 với tổng tiền tố của dãy còn lại. Nếu cả hai khác 0, ta có

$$h_i = s_f g_i + s_g f_i + f_i g_i,$$

rồi cập nhật s_f, s_g .

4.4.4. Tối ưu MAX-convolution bằng hợp nhất cây đoạn. Nếu trong DP trên cây có

$$D_i \leftarrow D_i \otimes D_v,$$

và mỗi D_i ban đầu chỉ có $O(1)$ vị trí khác 0, thì từ định nghĩa thấy vị trí k của kết quả khác 0 chỉ khi $f_k \neq 0$ hoặc $g_k \neq 0$. Vì vậy tập vị trí khác 0 của D_i luôn nằm trong hợp của các tập vị trí ban đầu thuộc cây con. Ta chỉ cần dùng cây đoạn mở nút động để lưu các vị trí khác 0.

Khi hợp nhất hai cây đoạn tương ứng với $F = D_i$ và $G = D_v$, duy trì hai tổng tiền tố s_f, s_g . Nếu một bên rỗng, đánh tag nhân với tổng tiền tố bên kia và trả về cây con đó; nếu cả hai có nút, đệ quy vào con trái rồi con phải và cập nhật. Quá trình có cùng độ phức tạp với hợp nhất cây đoạn thường, $O(n \log n)$.

4.4.5. Chuyển vị của cách tối ưu trên. Với DP có thêm mảng C_i :

$$D_i \leftarrow D_i \otimes D_v, \quad C_i \leftarrow C_i \otimes D_v + C_v \otimes D_i,$$

xem D là hằng số và bài toán là biến đổi tuyến tính theo C . Xét riêng

$$C_i \leftarrow C_i \otimes D_v.$$

Khi đảo quá trình hợp nhất, phải tách phần thuộc v khỏi cây đoạn đã hợp nhất. Viết $C_i = F, D_v = G$. Vì G là hằng số, tổng s_g giữ nguyên ý nghĩa; chỉ cần xử lý s_f theo chiều ngược.

Khi thăm nút pos được hợp nhất từ pos_1, pos_2 :

- nếu pos rỗng, trả về;

- nếu pos_1 không rỗng và pos_2 rỗng, với mọi f_i trong đoạn của pos_1 , thực hiện $f_i \leftarrow s_g f_i + s_f$;
- nếu pos_1 rỗng và pos_2 không rỗng, phép toán không liên quan đến F trong nhánh này;
- nếu cả hai không rỗng, xử lý hai con theo thứ tự ngược với lúc hợp nhất, rồi cập nhật nút hiện tại.

Nếu đi từ chuyển vị thô của MAX-convolution rồi mới tối ưu bằng cây đoạn, ta cũng có thể thu được kết quả tương tự, nhưng cần chứng minh thêm rằng chỉ những vị trí nằm trong tập đang được cây đoạn duy trì mới có ý nghĩa.

5. Ví dụ

5.1. Ví dụ 1

Nguồn. NOI2024 D2T2, *Đăng sơn*.

Tóm tắt đề. Cho cây gốc 1, cha của đỉnh i là p_i , độ sâu d_i là số cạnh từ i tới 1. Mỗi đỉnh có tham số l_i, r_i, h_i hạn chế đường leo núi hợp lệ. Một đường leo núi từ x là dãy

$$a_1 = x, a_2, \dots, a_m = 1$$

sao cho mỗi bước hoặc nhảy tới tổ tiên cấp k của đỉnh hiện tại với $l_{a_i} \leq k \leq r_{a_i}$, hoặc đi theo một quan hệ cha đặc biệt trong đề; đồng thời các lần nhảy phải thỏa ràng buộc về độ sâu h_i . Gọi f_x là số đường leo hợp lệ từ x tới 1; cần tính $f_2, \dots, f_n$.

Đặt

$$A = (f_2, f_3, \dots, f_n)^T, \quad a = (1),$$

tức cần tính $b = Aa$. Xét bài toán chuyển vị: cho hệ số

$$a' = (v_2, v_3, \dots, v_n)^T,$$

tính $b' = A^T a'$. Diễn giải thực tế là: mỗi đường leo hợp lệ có trọng số bằng trọng số v_s của điểm xuất phát s ; cần tính tổng trọng số của mọi đường leo hợp lệ.

Vì các điểm được “nhảy tới” luôn là tổ tiên của điểm nhảy trước đó, đặt g_x là tổng trọng số của mọi đường hiện đang nhảy tới x . Dùng thêm $c_{i,j}$ là số đường trong cây con i có thể nhảy lên tổ tiên ở độ sâu j , và $t_{i,j}$ là tổng trọng số của các đường như vậy. Nếu S_x là tập con của x , có dạng chuyển:

$$g_x = v_x + \sum_{i \in S_x} t_{i,d_x},$$

$$c_{x,j} = \begin{cases} [d_x - r_x \leq j \leq d_x - l_x] + \sum_{i \in S_x} c_{i,j}, & j < d_x - h_x, \\ 0, & j \geq d_x - h_x, \end{cases}$$

$$t_{x,j} = \begin{cases} g_x + \sum_{i \in S_x} t_{i,j}, & j < d_x - h_x, \\ 0, & j \geq d_x - h_x. \end{cases}$$

Có thể dùng hợp nhất cây đoạn để duy trì các mảng c, t , đạt $O(n \log n)$.

Khi chuyển vị quá trình trên, mảng c chỉ đóng vai trò hệ số nên có thể rollback trực tiếp. Bắt đầu từ $d_{1,0} = 1$, đảo các thao tác tại đỉnh x :

$$g_x \leftarrow \sum_{j < d_x - h_x} c_{x,j} t_{x,j},$$

rồi đặt $f_x \leftarrow g_x$, $t_{x,d_x} \leftarrow g_x$, và truyền $t_{x,j}$ xuống các con. Kết quả cuối cùng là mảng f . Cách nhìn chuyển vị giúp tránh việc phải tự thiết kế trực tiếp một DP khó tối ưu theo các hệ số c .

5.2. Ví dụ 2

Tóm tắt đề. Cho một cây, đỉnh i có trọng số p_i . Trọng số của một đồ thị con liên thông là giá trị lớn nhất trong các p_i của nó. Với mỗi đỉnh k , cần tính tổng trọng số của mọi đồ thị con liên thông chứa k .

Gọi $a_{i,j}$ là số đồ thị con liên thông chứa i và có trọng số lớn nhất bằng j , $A = (a_{i,j})$, và

$$a = (1, 2, \dots, n)^T.$$

Bài toán gốc là tính $b = Aa$. Xét chuyển vị với hệ số

$$a' = (v_1, v_2, \dots, v_n)^T.$$

Khi đó bài toán trở thành: mỗi đỉnh có trọng lượng v_i , trọng lượng của một đồ thị con liên thông là tổng trọng lượng các đỉnh của nó; với mỗi j , cần tính tổng trọng lượng của mọi đồ thị con liên thông có trọng số lớn nhất bằng j .

Bài toán chuyển vị này có DP tự nhiên. Gốc cây tại 1, gốc của một đồ thị con liên thông là đỉnh có độ sâu nhỏ nhất trong nó. Đặt $f_{i,j}$ là số đồ thị con liên thông có gốc i và trọng số lớn nhất j , còn $g_{i,j}$ là tổng trọng lượng của các đồ thị con đó. Ban đầu

$$F_i = x^{p_i}, \quad G_i = v_i x^{p_i}.$$

Khi gộp một con v vào i :

$$F_i \leftarrow F_i \otimes (1 + F_v),$$

$$G_i \leftarrow G_i \otimes (1 + F_v) + G_v \otimes F_i.$$

Sau mỗi lần gộp, cộng G_i vào đáp án theo trọng số. Các phép $\otimes$ là MAX-convolution và có thể tối ưu bằng hợp nhất cây đoạn, nên bài toán chuyển vị giải được trong $O(n \log n)$.

Để quay lại bài toán gốc, đảo quá trình trên. Các chuyển của F không phụ thuộc hệ số đầu vào nên chỉ cần rollback. Nếu đáp án chuyển vị nằm trong mảng y , thì với bài toán gốc cần tính

$$y = \sum_i ix_i.$$

Khi xử lý ngược tại đỉnh i :

- $G_i \leftarrow G_i + y$;
- tách các con theo thứ tự ngược: $G_v \leftarrow G_i \otimes^T F_i$, $G_i \leftarrow G_i \otimes^T (F_v + 1)$;
- sau khi tách xong, lấy đóng góp tại p_i để gán cho v_i .

Nhờ cách xử lý $\otimes^T$ bằng tách cây đoạn ở phần trước, bài toán gốc cũng được giải trong $O(n \log n)$. Với ví dụ này, nếu thiết kế DP trực tiếp cho bài toán gốc thì trạng thái, chuyển và tối ưu chuyển đều kém tự nhiên hơn đáng kể; chuyển vị đem lại lợi thế rõ ràng.

6. Tổng kết

Bài viết giới thiệu nguyên lý chuyển vị, khảo sát chuyển vị của một số thao tác trên cây đoạn, rồi dùng hai ví dụ để minh họa ứng dụng trong các bài toán quy hoạch động có thể mô tả thành biến đổi tuyến tính. Với những bài toán cần hợp nhất hoặc tách cây đoạn để tối ưu chuyển, nguyên lý chuyển vị có thể giảm đáng kể độ khó khi thiết kế trạng thái, thiết kế chuyển và chứng minh độ phức tạp, đồng thời vẫn đạt được bậc $O(n \log n)$ quen thuộc của hợp nhất cây đoạn.

7. Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia đã giúp đỡ và chỉ đạo. Xin cảm ơn thầy Xiang Qizhong và thầy Hu Weidong đã quan tâm, hướng dẫn. Xin cảm ơn gia đình đã ủng hộ và động viên tác giả. Xin cảm ơn Zhang Ruichen, Wu Ruiqi và các bạn khác đã đọc kiểm và góp ý cho bài viết.

Tài liệu tham khảo

1. A. Bostan, G. Lecerf, and E. Schost. Tellegen's principle into practice. *Proceedings of the 2003 International Symposium on Symbolic and Algebraic Computation*, pages 37–44, 2003.
2. Gilbert Strang. *Linear Algebra*, 5th edition. Tsinghua University Press, 2019.
3. Chen Yu. Giới thiệu đơn giản về nguyên lý chuyển vị. Tập luận văn đội tuyển quốc gia 2020, 2020.

Bài 5. Bàn về ứng dụng cơ sở tuyến tính trong OI

Chen Xinyang, Trường Trung học số 2 Hàng Châu

Tóm tắt

Cơ sở tuyến tính là công cụ mạnh trong các bài toán về xor của dãy con, đồng thời có nhiều mở rộng hữu dụng trong thi tin học. Bài viết trình bày cách xây dựng cơ sở tuyến tính và các ứng dụng cơ bản, sau đó hệ thống hóa một số kỹ thuật tối ưu thường gặp. Phần sau giới thiệu cơ sở tuyến tính trực giao, cách lấy giao của các không gian sinh bởi xor, và cuối cùng thảo luận cơ sở tuyến tính có hỗ trợ xóa. Với biến thể có xóa, bài viết đưa ra hai cách cài đặt, trong đó có một phiên bản giữ được đầy đủ tính chất “tiền tố”, nhằm xử lý một số bài toán trước đây khó giải bằng kỹ thuật quen thuộc.

1. Dẫn nhập

Trong bài viết này, ký hiệu $\oplus$ chỉ phép xor theo bit. Với một dãy số nguyên không âm S , giả sử chỉ cần xét m bit thấp. Ta gọi dãy T là một cơ sở tuyến tính của S nếu nó thỏa ba điều kiện sau.

1. Các giá trị xor của mọi dãy con của T đôi một khác nhau, kể cả dãy con rỗng.
2. Tập các giá trị xor thu được từ dãy con của S trùng với tập các giá trị xor thu được từ dãy con của T .
3. Nếu bỏ các phần tử bằng 0, vị trí bit cao nhất bằng 1 của các phần tử trong T giảm nghiêm ngặt.

Nhìn theo đại số tuyến tính, đây chính là một cơ sở của không gian sinh bởi các véc-tơ bit trên trường $\mathbb{F}_2$. Tuy vậy, khi dùng trong OI, ta thường diễn đạt bằng thao tác xor để thuận tiện hơn.

Bố cục của bài viết như sau. Phần 2 nhắc lại cách xây dựng và các ứng dụng cơ bản. Phần 3 bàn về các kỹ thuật tối ưu khi phải xây dựng hoặc truy vấn nhiều cơ sở tuyến tính. Phần 4 giới thiệu cơ sở tuyến tính trực giao và phép lấy giao. Phần 5 tập trung vào cơ sở tuyến tính có hỗ trợ xóa, gồm một cách cài đặt đơn giản nhưng còn khuyết điểm và một cách cài đặt giữ đầy đủ tính chất tiền tố.

2. Xây dựng và ứng dụng cơ bản

Mấu chốt của cơ sở tuyến tính là phép biến đổi bảo toàn toàn bộ tập giá trị xor của dãy con. Nếu chọn hai vị trí $i \neq j$ và thay S_j bằng $S_i \oplus S_j$, thì dãy mới tương đương với dãy cũ: với mỗi dãy con ở một bên, ta

luôn đổi được sang một dãy con ở bên kia có cùng giá trị xor. Lặp lại thao tác khử bit giống khử Gauss, ta đưa dãy ban đầu về một dãy tương đương có các bit cao nhất khác nhau.

Khi chèn từng phần tử, ta duy trì mảng w_p , trong đó w_p là phần tử cơ sở có bit cao nhất bằng p , và một biến c đếm số phần tử phụ thuộc tuyến tính bị khử thành 0. Để chèn x , duyệt các bit từ cao xuống thấp. Nếu bit j của x bằng 0, bỏ qua. Nếu $w_j = 0$, đặt $w_j = x$ và dừng. Nếu $w_j \neq 0$, gán $x \leftarrow x \oplus w_j$ rồi tiếp tục. Nếu cuối cùng $x = 0$, tăng c . Độ phức tạp là $O(|S|m)$, bộ nhớ $O(m)$.

Một hệ quả quan trọng là các phần tử cơ sở khác 0 sinh ra đúng 2^r giá trị xor khác nhau, với r là số phần tử cơ sở. Nếu trong quá trình xây dựng có c phần tử bị khử thành 0, thì mỗi giá trị biểu diễn được xuất hiện đúng 2^c lần khi đếm theo dãy con của dãy gốc.

Giá trị xor lớn nhất. Sau khi có cơ sở, để tìm giá trị xor lớn nhất của một dãy con, khởi tạo $ans = 0$. Duyệt j từ cao xuống thấp và thay ans bằng $\max(ans, ans \oplus w_j)$. Cách này cũng áp dụng khi cần tối đa hóa $v \oplus x$, với x là xor của một dãy con: chỉ cần khởi tạo $ans = v$.

Đếm số dãy con có xor bằng một giá trị. Với giá trị cần kiểm tra x , thử khử x bằng cơ sở hiện có. Nếu khử được về 0, x biểu diễn được và số dãy con có xor bằng x là 2^c . Nếu không, đáp án bằng 0.

Đếm số giá trị nhỏ hơn một ngưỡng. Sau khi bỏ phần tử phụ thuộc, các giá trị xor sinh bởi cơ sở đôi một khác nhau. Ta duyệt bit từ cao xuống thấp, giữ tiền tố hiện tại và so sánh với lim . Khi bit của lim bằng 1, có thể cộng số lựa chọn làm tiền tố nhỏ hơn ngay tại bit đó; phần còn lại được chọn tùy ý từ các cơ sở ở bit thấp hơn. Nhánh còn lại buộc phải giữ tiền tố bằng lim và tiếp tục khử. Cuối cùng nhân thêm 2^c nếu đếm dãy con của dãy gốc. Cùng ý tưởng này cũng cho phép tìm giá trị xor nhỏ thứ k .

Đường đi xor lớn nhất trong đồ thị vô hướng. Bài WC2011 “Đường đi xor lớn nhất” là ví dụ kinh điển. Chọn một đường đi bất kỳ từ 1 đến n , có xor là v . Mỗi khi chèn thêm một chu trình vào đường đi, hai đầu mút không đổi và giá trị đường đi bị xor thêm giá trị xor của chu trình đó. Vì thế đáp án là giá trị lớn nhất của $v \oplus x$, trong đó x thuộc không gian sinh bởi các xor chu trình.

Không cần liệt kê mọi chu trình. Chọn một cây khung, đặt d_u là xor trên đường từ gốc đến u trong cây. Với mỗi cạnh ngoài cây (u, v, w) , giá trị $d_u \oplus d_v \oplus w$ là xor của chu trình cơ bản tương ứng. Các chu trình cơ bản này sinh ra toàn bộ không gian chu trình, nên chỉ cần đưa chúng vào cơ sở tuyến tính rồi tối đa hóa v . Độ phức tạp là $O(|V| + |E|m)$.

3. Các kỹ thuật tối ưu thường gặp

Nếu S' là cơ sở của S , T' là cơ sở của T , thì cơ sở của dãy ghép $S + T$ có thể xây từ $S' + T'$. Vì mỗi cơ sở có nhiều nhất m phần tử, phép gộp tốn $O(m^2)$. Đây là nền tảng cho cây đoạn lưu cơ sở tuyến tính trên mỗi nút: cập nhật một điểm và truy vấn một đoạn đều làm việc qua các phép gộp, đạt độ phức tạp kiểu $O(\log n \cdot m^2)$ mỗi thao tác. Nhược điểm là hằng số và nhân tử m^2 khá nặng.

Một kỹ thuật khác là đổi dãy đang duy trì sang một dãy tương đương thuận lợi hơn. Chẳng hạn với bài toán cập nhật xor trên đoạn và hỏi xor dãy con lớn nhất trên đoạn, đặt $b_i = a_i \oplus a_{i-1}$. Khi đó đoạn $a_1, \dots, a_r$ tương đương với phần tử a_l cùng dãy $b_{l+1}, \dots, b_r$, còn cập nhật xor trên đoạn chỉ làm đổi b_l và b_{r+1} . Cây đoạn trên dãy b , kèm thêm thao tác chèn a_l khi trả lời, cho lời giải tự nhiên hơn. Với cập nhật theo bước k , có thể dùng $b_i = a_i \oplus a_{i-k}$ tương tự.

Cơ sở tuyến tính tiền tố. Khi xây cơ sở từ trái sang phải, ta có thể ghi lại thời điểm mỗi phần tử cơ sở được đưa vào. Tổng quát hơn, gán cho mỗi giá trị a_i một độ ưu tiên t_i . Trong mảng cơ sở, ngoài w_j còn

lưu độ ưu tiên p_j của phần tử đang chiếm bit j . Khi chèn một cặp (t, x) , nếu gặp một bit đã có w_j nhưng độ ưu tiên của x cao hơn, ta hoán đổi x với w_j rồi tiếp tục khử phần còn lại. Nhờ đó cơ sở giữ các phần tử có độ ưu tiên cao nhất có thể.

Ứng dụng tiêu biểu là CF1100F *Ivan and Burgers*: với mỗi truy vấn tính $[l, r]$, sau khi đã xử lý đến r , chỉ cần lấy các phần tử cơ sở có độ ưu tiên ít nhất l . Cách này trả lời mọi truy vấn trong $O((n + q)m)$. Trên cây, bài SCOI2016 “Lucky Numbers” dùng độ sâu làm độ ưu tiên để lấy cơ sở trên đường tổ tiên; đường đi bất kỳ được tách thành hai đoạn quanh LCA rồi gộp. Với tập động nhưng biết trước thời điểm xóa, có thể dùng thời điểm xóa làm độ ưu tiên để tránh thao tác xóa thật sự. Tuy nhiên cách này chỉ dùng tốt trong bối cảnh offline và khi bài toán không cần thêm một thứ tự ưu tiên khác.

Khử tiếp các bit thấp. Sau khi có cơ sở, ta thường khử tiếp các bit thấp khỏi các phần tử cơ sở cao: duyệt i từ thấp lên cao, nếu $w_i \neq 0$ thì với mọi $j > i$ có bit i bằng 1, gán $w_j \leftarrow w_j \oplus w_i$. Khi đó mỗi bit thấp chỉ xuất hiện trong phần tử cơ sở tương ứng của nó. Dạng chuẩn hóa này giúp một số hàm trên cơ sở trở nên tuyến tính hơn.

Ví dụ, đặt $g(x, S)$ là giá trị lớn nhất của $x \oplus$ xor dãy con của S , và $g'(x, S)$ là giá trị nhỏ nhất. Sau khi chuẩn hóa, g' có tính tuyến tính theo xor:

$$g'(x_1 \oplus \dots \oplus x_k, S) = g'(x_1, S) \oplus \dots \oplus g'(x_k, S).$$

Ngoài ra $g(x, S) \oplus g'(x, S)$ bằng xor của các phần tử cơ sở. Trong bài “Xor Master”, tính chất này cho phép duy trì các giá trị đã biến đổi khi cơ sở của một tập S_0 mở rộng, tránh cập nhật lại toàn bộ với chi phí $O(nm)$ mỗi lần.

Kết hợp cơ sở tiền tố với cấu trúc dữ liệu. Với bài toán duy trì nhiều dãy $S_1, \dots, S_n$, hỗ trợ thêm phần tử vào một dãy và hỏi giá trị xor lớn nhất từ dãy ghép $S_l + \dots + S_r$, có thể dùng cây đoạn. Ở các nút con trái lưu cơ sở hậu tố, ở các nút con phải lưu cơ sở tiền tố. Khi truy vấn, tách đoạn tại nút đầu tiên mà hai đầu rơi sang hai phía, rồi gộp cơ sở hậu tố bên trái và cơ sở tiền tố bên phải. Cập nhật thêm phần tử tốn $O(m \log n)$, truy vấn tốn $O(m^2)$. Cách làm này cho thấy sức mạnh của cơ sở tiền tố, nhưng chưa xử lý được sửa đổi tùy ý bên trong dãy; vấn đề đó được đưa sang phần về cơ sở có xóa.

4. Cơ sở tuyến tính trực giao

Xét dãy điểm-giá trị $v(S)$ độ dài 2^m , trong đó $v(S)_i$ là số dãy con của S có xor bằng i . Phép ghép hai dãy tương ứng với tích chập xor:

$$c_k = \sum_{i \oplus j = k} a_i b_j.$$

Với hai số m -bit, định nghĩa

$$i \odot j = \text{popcount}(i \& j) \bmod 2.$$

Biến đổi Walsh–Hadamard cho xor là

$$\hat{a}_i = \sum_j (-1)^{i \odot j} a_j.$$

Nó biến tích chập xor thành nhân theo điểm.

Với một phần tử đơn x , dãy điểm-giá trị có hai lựa chọn: không lấy hoặc lấy x . Vì vậy

$$\widehat{v(S)}_k = \prod_{x \in S} (1 + (-1)^{x \odot k}).$$

Giá trị này bằng $2^{|S|}$ khi k trực giao với mọi phần tử của S , và bằng 0 trong các trường hợp còn lại. Do trực giao với S tương đương trực giao với bất kỳ cơ sở nào của S , ta chỉ cần xét cơ sở.

Điều này hữu ích khi số véc-tơ trực giao ít. Trong CF1336E2, cần đếm dãy con theo số bit 1 của xor. Nếu cơ sở ban đầu nhỏ, ta liệt kê các xor do cơ sở sinh ra. Nếu cơ sở lớn, không gian trực giao nhỏ hơn; ta liệt kê cơ sở trực giao rồi dùng FWT để suy ra đáp án. Cân bằng hai phía cho độ phức tạp khoảng $O(nm + 2^{m/2} + m^3)$.

Dựng cơ sở trực giao. Trước hết chuẩn hóa cơ sở bằng cách khử các bit thấp. Với mỗi vị trí bit i mà $w_i = 0$, bit i của một véc-tơ trực giao được chọn tự do. Với mỗi $w_i \neq 0$, bit i của véc-tơ trực giao bị xác định duy nhất để tích vô hướng với w_i bằng 0. Vì vậy, với mỗi ô trống i , tạo một véc-tơ

$$w'_i = 2^i + \sum_{\substack{j > i \\ \text{bit } i \text{ của } w_j \text{ bằng } 1}} 2^j.$$

Các w'_i sinh đúng toàn bộ không gian trực giao. Chi phí dựng là $O(m^2)$, hoặc $O(sm)$ nếu chỉ có s vị trí cần xét.

Giao của nhiều cơ sở. Muốn kiểm tra một giá trị x có biểu diễn được đồng thời bởi nhiều dãy $S_1, \dots, S_k$ hay không, ta cần giao của các không gian sinh. Dựa trên quan hệ $(A \cap B)^\perp = A^\perp + B^\perp$, có thể lấy trực giao từng cơ sở, gộp tất cả cơ sở trực giao, rồi lấy trực giao lần nữa. Nói ngắn gọn: giao bằng trực giao của hợp các không gian trực giao.

Trong UOJ698, với các tập S_i , mỗi truy vấn hỏi chỉ số nhỏ nhất i sao cho S_i không có dãy con xor bằng x . Nếu S'_i là cơ sở trực giao của S_i , thì x không biểu diễn được bởi S_i khi và chỉ khi có một phần tử của S'_i không trực giao với x . Ta quét các i , chèn các phần tử của S'_i vào một cơ sở và ghi lại thời điểm chúng xuất hiện. Truy vấn trở thành tìm thời điểm đầu tiên có phần tử không trực giao với x , xử lý trong $O(m)$ sau tiền xử lý. Tổng độ phức tạp là $O(nm^2 + \sum |S_i|m + qm)$.

5. Cơ sở tuyến tính có xóa

Các phần trước thường tránh xóa thật sự: hoặc chuyển bài toán thành gộp cơ sở trên cây đoạn, hoặc dùng thời điểm xóa làm độ ưu tiên trong một bài toán offline. Những chuyển hóa này khá mong manh. Khi bài toán bắt buộc online, hoặc khi độ dài bit lớn khiến phép gộp $O(m^2)$ trở thành nút thắt, ta cần đối mặt trực tiếp với thao tác xóa.

Một phiên bản còn khuyết điểm. Xét bài toán động trên các cặp (t_i, a_i) : chèn một cặp, xóa một cặp đã có, và với một ngưỡng lim , hỏi số giá trị xor khác nhau sinh bởi các a_i có $t_i \leq \text{lim}$, modulo 998244353. Ở đây $n, m \leq 2000$. Cơ sở tiền tố thông thường không đủ, vì điều kiện vừa theo thời gian tồn tại, vừa theo độ ưu tiên, tạo thành một truy vấn hai chiều.

Ta đánh số phần tử theo thứ tự chèn. Mỗi giá trị a_i có một biểu diễn resp_i bằng một bitset trên các phần tử chuẩn. Mỗi ô cơ sở w_j cũng lưu wresp_j , tức biểu diễn của w_j bằng các phần tử chuẩn. Nếu xóa một phần tử không nằm trong cơ sở chuẩn, chỉ cần bỏ nó. Nếu xóa một phần tử chuẩn i , ta tìm phần tử ngoài chuẩn có độ ưu tiên nhỏ nhất phù hợp để thay thế, cụ thể là một phần tử j có biểu diễn chứa i . Khi tìm được j , dùng các bitset để thay i bằng j trong mọi biểu diễn; nếu không tìm được, loại i khỏi cơ sở và cập nhật các biểu diễn liên quan. Thao tác chèn xử lý đối xứng: phần tử mới có thể đẩy một phần tử ưu tiên thấp hơn ra khỏi cơ sở.

Chi phí mỗi chèn hoặc xóa là $O((n+m)^2/w)$, với w là độ rộng từ máy, vì phải thao tác trên các bitset dài. Cách cài đặt này online và hữu ích trong một số bài, nhưng có hai khuyết điểm: chi phí phụ thuộc

vào số phần tử đang duy trì, và nó chỉ giữ được một phần tính chất tiền tố, chưa đủ để thay thế hoàn toàn cơ sở tiền tố trong mọi cấu trúc dữ liệu.

Kết hợp phiên bản có xóa với cây đoạn. Quay lại bài toán duy trì mảng a , hỗ trợ sửa một điểm $a_x = y$ và hỏi giá trị xor lớn nhất của dãy con trong đoạn. Cách ở phần 3 dùng cây đoạn và gộp cơ sở tiền tố $O(m^2 \log n)$. Với cơ sở có xóa ở trên, ta có thể tối ưu vì tại mỗi nút cây đoạn chỉ cần duy trì các phần tử chuẩn của hai con; số lượng phần tử chuẩn không vượt quá $2m$.

Cụ thể, mỗi nút lưu hai cơ sở tiền tố có xóa, một cho hướng xuôi và một cho hướng ngược. Ở lá, cơ sở chuẩn có nhiều nhất một phần tử. Ở nút trong, cơ sở được xây từ các phần tử chuẩn của hai con. Khi sửa một điểm, thay đổi ở lá tạo ra một số sự kiện chèn/xóa trong cơ sở chuẩn; các sự kiện này được truyền lên cha. Vì mỗi nút chỉ duy trì $O(m)$ phần tử, mỗi tầng chỉ phát sinh lượng nhỏ thao tác. Truy vấn vẫn tách đoạn ở điểm chia đầu tiên như phần 3: lấy cơ sở hậu tố từ bên trái và cơ sở tiền tố từ bên phải rồi gộp để trả lời.

Để giảm chi phí dựng và bộ nhớ, có thể chặn đáy cây bằng phân khối: mọi nút có độ dài đoạn không quá m được xem như lá giả, bỏ các hậu duệ của nó. Khi sửa bên trong lá giả, dựng lại trực tiếp trong $O(m^2)$, rồi truyền thay đổi lên trên. Khi truy vấn rơi hoàn toàn trong một lá giả, dựng cơ sở bằng vét cạn trên đoạn ngắn. Nhờ vậy, bài toán có lời giải online $O(nm + qm(\log n + m))$ thời gian và $O(n)$ bộ nhớ, tốt hơn phương pháp cây đoạn truyền thống về bậc độ phức tạp, dù ý tưởng và cài đặt phức tạp hơn.

Phiên bản giữ đầy đủ tính chất tiền tố. Phần này đưa ra một cài đặt mạnh hơn. Mục tiêu là dù chèn các cặp theo thứ tự nào, hình dạng cuối cùng của cơ sở vẫn giống hệt cơ sở thu được khi chèn $a_1, a_2, \dots, a_n$ theo đúng thứ tự chỉ số bằng thuật toán cơ bản ở phần 2. Tính “giống hệt” này rất quan trọng: khi xóa, ta có thể xem phần tử bị xóa như phần tử vừa được chèn cuối cùng, rồi đảo ngược quá trình chèn đó.

Để mô tả trạng thái, ngoài w_i và $wresp_i$, đặt idx_i là chỉ số cao nhất xuất hiện trong biểu diễn $wresp_i$; nếu $w_i = 0$ thì xem $idx_i = +\infty$. Khi chèn một phần tử ở vị trí p , duy trì ba đại lượng: cur là giá trị đang xử, $cresp$ là biểu diễn của nó bằng các phần tử gốc, và q là chỉ số đại diện hiện tại. Ban đầu $cur = a_p$, $cresp = 2^p$, $q = p$. Duyệt bit từ cao xuống thấp. Nếu bit hiện tại của cur bằng 0, ta vẫn có thể phải dùng $cresp$ để loại q khỏi các biểu diễn $wresp$ nhằm giữ ràng buộc mạnh hơn. Nếu bit bằng 1, ta so sánh q với idx_i ; khi cần, hoán đổi cur với w_i , $cresp$ với $wresp_i$, rồi xor để khử bit. Nếu sau một bước $q = +\infty$, có thể dừng.

Khi chèn xong, có hai khả năng. Nếu $q = +\infty$, phần tử p chỉ được thêm vào cơ sở chuẩn và không có phần tử chuẩn nào bị loại. Nếu $1 \leq q \leq n$, thì p đi vào cơ sở chuẩn còn q bị đẩy ra; khi đó cần dùng $cresp$ để sửa mọi biểu diễn có chứa q .

Thao tác xóa được thực hiện bằng cách khôi phục trạng thái trước lần chèn cuối. Trước hết phải xác định lần chèn đó kết thúc ở trường hợp nào. Nếu có một phần tử ngoài cơ sở có biểu diễn chứa p , lấy phần tử có chỉ số nhỏ nhất như vậy làm q ; khi đó có thể khôi phục $cur = 0$ và suy ra $cresp$ từ biểu diễn của q . Nếu không tồn tại, ta ở trường hợp $q = +\infty$ và cần tìm vị trí bit mà quá trình chèn đã dừng.

Dấu hiệu quan trọng là bit p trong các biểu diễn. Trước khi chèn a_p , không biểu diễn nào chứa bit này; trong quá trình chèn, bit p chỉ lan sang các $wresp$ thông qua hoán đổi hoặc qua bước khử đặc biệt ở nhánh bit 0. Vì thế khi đảo ngược từng bit, ta dùng việc $wresp_i$ có chứa p hay không để biết bước trước đó có hoán đổi hoặc có khử bằng $cresp$ hay không. Trường hợp khó nhất là phân biệt bit hiện tại của cur trước bước đó bằng 0 hay 1; bài viết chỉ ra có thể quyết định bằng cách kiểm tra các bit cao trong $cresp$ và quan hệ với idx_i . Nhờ các dấu hiệu này, mọi bước chèn đều đảo ngược được một cách xác định.

Cả chèn và xóa đều bị chi phối bởi $O(n)$ phép xor trên bitset dài n , nên mỗi sửa điểm tốn $O(n^2/w)$. Sau khi duy trì được dạng cơ sở mạnh hơn cơ sở tiền tố thông thường, truy vấn các thông tin kinh điển của cơ sở tuyến tính cũng xử lý trong $O(n^2/w)$. Tính cả thời gian xây dựng, ví dụ trong bài viết được

giải online trong $O((n + q)n^2/w)$.

Tổng kết phần cơ sở có xóa. Phiên bản ở mục đầu đơn giản hơn nhưng chỉ giữ một phần tính tiền tố. Phiên bản sau phức tạp hơn, song giữ được đầy đủ tính tiền tố và vì vậy dùng được trong những bài toán động khó hơn. Tác giả nhận xét rằng chi phí của cơ sở có xóa khó tránh khỏi việc phụ thuộc vào số phần tử được duy trì; vì thế khi số phần tử lớn hơn nhiều so với độ dài bit, các kỹ thuật truyền thống đôi khi vẫn phù hợp hơn. Tuy nhiên, trong những bài mà số phần tử cùng bậc với độ dài bit và bắt buộc online, hướng tiếp cận này mở ra các lời giải mới.

6. Tổng kết

Bài viết lần lượt trình bày cách xây dựng cơ sở tuyến tính, các ứng dụng cơ bản, những kỹ thuật tối ưu thường dùng, cơ sở trực giao, phép lấy giao và cơ sở tuyến tính có hỗ trợ xóa. Ba phần đầu chủ yếu hệ thống hóa các kỹ thuật đã quen thuộc trong thi tin học; phần cuối đưa ra thêm một số lý thuyết và cách cài đặt mới cho hướng cơ sở tuyến tính có xóa, qua đó tối ưu một số bài toán cổ điển. Tác giả hy vọng các ý tưởng này gợi mở thêm nhiều bài toán và mở rộng thú vị liên quan đến cơ sở tuyến tính.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Peng Sijin và Yang Yaoliang đã hướng dẫn. Xin cảm ơn cha mẹ, nhà trường, thầy Li Jian, cùng các bạn học đã giúp đỡ. Xin cảm ơn Zhang Shaojia và Wang Suyi đã trao đổi, thảo luận và đọc kiểm bài viết.